

1)Intro Embedded systems:

Embedded = essere parte integrale di un sistema più grande. Il sistema + grande è l'embedding system.

Sono sistemi con applicazioni specifiche, spesso devono essere responsivi in real time.

In genere questi sistemi consistono nel : ricevere input dall'esterno, comunicare internamente con altri sistemi, magari offrire un'interfaccia utente, e poi fornire risultati come output.

Tipicamente sono implementati usando dei microcontrollori (MCUs).

Differenza tra microcontrollore e microprocessore:

Di base entrambi hanno un core che esegue le istruzioni.

Microprocessore (CPU): memoria e I/O esterni. Lui fa solo fetch, decode, execute. Da solo non basta a far funzionare un sistema. Usato per general-purpose computing, infatti i microprocessori usano una stessa logica hardware per svolgere tante funzioni diverse.

MicroControllore(MCU): è un chip che **integra al suo interno** non solo il microprocessore, ma anche memoria (RAM, ROM/Flash) e periferiche di I/O. È quindi un sistema completo "tutto in uno" progettato per applicazioni embedded. In genere i microcontrollori sono preferiti per gli embedded systems perchè offrono costi minori di produzione, facilitano il porting, hanno bassi consumi. **Un microcontrollore contiene al suo interno un microprocessore**, oltre a memoria e periferiche.

Ci sono varie sfide da affrontare nella progettazione di un embedding system:

Come testare se funziona correttamente; specialmente come testo le funzioni real-time? Il testing deve in tanti casi avvenire con la macchina in cui il computer lavora.

Come posso aggiungere nuove feature? E' difficile controllare/osservare il funzionamento (no schermi, no tastiere.. bisognerebbe guardare i valori dei segnali elettrici).

Un buon sistema embedded deve rispondere in real-time, gestire da solo gli errori + comuni (fault handling), performare + attività concorrentemente.

Embedded Software: Di solito si programma in C e in alcuni casi in assembly. Tipicamente non c'è nessun OS, ma solo dei semplici scheduler. Se viene usato un OS , comunque questo è un real time OS (RTOS). Spesso l'hardware e il software vengono sviluppati in parallelo.

Un programma embedded deve fornire un output richiesto entro una certa **deadline = il tempo entro cui un certo calcolo deve essere completato**. Non ce ne frega se l'output è giusto se non viene prodotto entro la deadline.

Sistema IoT = un sistema embedded collegato in rete: un vasto ecosistema di dispositivi che comunicano tra loro attraverso una rete.

2)Architettura ARM Cortex-M4

I microcontroller che vedremo hanno al loro interno dei processori ARM Cortex -M4.

ARM (Advanced Risc Machines)

E' un'architettura **RISC**, che è perfetta per i sistemi embedded per la semplicità del set di istruzioni. Infatti noi cerchiamo power efficiency, quindi RISC è ormai un defacto a questi scopi. Le istruzioni semplici possono essere messe facilmente in pipeline. Queste prestazioni producono anche un basso consumo energetico (molto importante durata batteria nei sistemi embedded).

Le ISA RISC sono di tipo load/store, ciò significa che si può solo operare sui registri, non direttamente nelle locazioni di memoria. Prima tramite una load carichiamo il dato nel registro, poi facciamo operazioni (write), e rimettiamo il risultato nel registro, e infine facciamo una store per rimettere il risultato in memoria.

Le singole compagnie possono: o comprare l'architectural license da ARM company, oppure la microarchitectural license..

L'**architettura** di un processore definisce le caratteristiche che devono essere vere per tutte le sue implementazioni.

La **microarchitettura**, o "implementazione", si riferisce al modo specifico in cui un'architettura viene realizzata. Diverse implementazioni della stessa architettura possono avere:

- **Velocità di clock** variabili.
- **Larghezze di bus** differenti.
- **Dimensioni della cache** diverse.
- E altri dettagli interni che influenzano le prestazioni e le caratteristiche fisiche del chip, pur rispettando il set di istruzioni e il modello del programmatore definiti dall'architettura. Ad esempio, un processore ARM è sviluppato considerando una delle architetture ARM esistenti

1. Licenza Microarchitetturale (RTL-level design): Le aziende possono utilizzare una delle varie licenze Cortex disponibili. Questo significa che ricevono il design già pronto a livello RTL (Register Transfer Level) del processore, come il Cortex-M4, e possono integrarlo nel loro SoC. Questa scelta è meno flessibile ma più rapida in termini di sviluppo, in quanto si basa su un design pre-validato da ARM.

2. Licenza Architetturale: L'azienda può sviluppare la propria microarchitettura basata sul set di istruzioni (ISA) di ARM. Questa opzione offre la massima flessibilità, consentendo all'azienda di progettare un core del processore completamente personalizzato per soddisfare esigenze specifiche di prestazioni, consumo energetico o dimensioni, pur mantenendo la compatibilità con il set di istruzioni ARM.

In sintesi, ARM progetta le architetture e licenzia i design ai partner semiconduttori, che poi aggiungono la propria proprietà intellettuale (IP) e producono il chip.

Ci sono 3 classi di processori ARM, ognuna con un suo scopo.

La famiglia Cortex-M (MicroController) offre soluzioni (serie di core) per microcontrollori deterministici.

Caratteristiche delle serie Cortex-M:

- Efficienza energetica -> batteria con più lunga vita
- Smaller code
- Costi più bassi, meno spazio fisico
- E' importante avere una grande community che dà supporto su un certo hardware, e questo è il caso.

Cortex M0 e M0+ : ottimizzato per semplici sensing e controlli

Cortex M3, Cortex-M4 e Cortex M7: Integrano il Digital Signal Processing (DSP) e l'accelerated floating point processing -> Esecuzione veloce di algoritmi.

Abbiamo visto che le differenti implementazioni (microarchitetture) di una singola architettura devono seguire delle stesse caratteristiche ma variare altro ad esempio le velocità di clock.

L'architettura ARM descrive i dettagli del set di istruzioni, del mapping di memoria e del programmatore.

Invece un processore ARM è sviluppato considerando una tra le architetture ARM.

C'è una continua evoluzione dell'architettura ARM (es: ARMv6, ARMv7E-M....).

Caratteristiche del processore Cortex-M4: la sua architettura è la Armv7E-M: è un'architettura Harvard.

Come il software, anche l'hardware può eseguire delle moltiplicazioni (molto + velocemente), e nel M4 in un ciclo si performato moltiplicazioni hardware. Inoltre supporta anche le divisioni hardware. Contiene anche estensioni DSP(Digital Signal processing).

Questo per consentire l'elaborazione **rapida ed efficiente in termini di consumo energetico** di algoritmi di Digital Signal Processing (DSP). Le funzioni tipiche del digital signal processing

includono la **rimozione del rumore** e la **selezione delle caratteristiche desiderate del segnale**.

Le floating point operation vengono fatte col software nel M0, ma nel M4 c'è una floating point unit che performa tale operazione direttamente sull'hardware.

Von Neumann Architecture vs Harvard Architecture

L'architettura **Von Neumann** si caratterizza per una **singola memoria condivisa per contenere sia i dati che le istruzioni**. Qui la CPU preleva un'istruzione dalla memoria, la decodifica e la esegue. Durante questo processo la stessa memoria viene utilizzata per accedere ai dati necessari all'istruzione.

I registri della CPU (PC, IR, general purpose...) immagazzinano i valori utilizzati internamente dal processore. **Svantaggi**: Il principale svantaggio è il cosiddetto "collo di bottiglia di Von Neumann", perchè sia le istruzioni che i dati devono passare attraverso lo stesso bus di memoria, e quindi non possono essere prelevati simultaneamente -> rallenta esecuzione del programma.

L'architettura **Harvard** si distingue per l'utilizzo di **memorie separate per i dati e per le istruzioni**.

Qui, il program counter (PC) punta alla memoria dei programmi e non alla memoria dei dati. Questo significa che il processore può accedere contemporaneamente a un'istruzione dalla memoria dei programmi e a un dato dalla memoria dei dati, in quanto dispongono di porte e bus separati.

Ciò ha il vantaggio degli accessi simultanei -> migliora prestazioni. Inoltre impossibile avere errori dati da programmi auto-modificanti.

La maggior parte dei processori DSP (progettati per elaborare segnali digitali in tempo reale) usano Harvard per le prestazioni.

Overview del processore Cortex-M4

Abbiamo visto che offre un efficiente signal processing, ad esempio le istruzioni "**single cycle multiply accumulate (MAC)**".

Determinismo: le task critiche e le interrupt routines possono essere servite velocemente in un numero conosciuto di cicli.

Usa **architettura Harvard**.

E' un processore RISC a **32-bit**, che opera sui registri attraverso un'architettura di tipo load/store.

Possiede una pipeline a 3 stadi (fetch, decode, execute), che include anche la funzionalità di **branch speculation**: Quando la pipeline incontra un'istruzione di salto condizionato, dovrebbe attendere che questa venga completata nella fase Execute prima di fetchare la prossima istruzione, ma questo creerebbe uno stallo; allora il processore specula/cerca di indovinare l'esito del salto per far continuare la pipeline.

Include una **floating point unit** opzionale (FPU) per elaborazione accelerata in virgola mobile. Supporta anche le modalità di riposo (sleep modes), con dei wake-up interrupts.

Il **Processor core** è la parte centrale del processore e contiene solo registri interni e ALU.

Il **Nested Vectored Interrupt Controller (NVIC)**: Gestisce automaticamente gli interrupt annidati, confrontando le priorità delle richieste di interrupt con il livello di priorità corrente.

Il **Wake-up Interrupt Controller (WIC)**: in applicazioni a basso consumo consente al microcontroller di entrare in **modalità sleep** e quando un interrupt viene rilevato risveglia il sistema su rilevamento di interrupt/eventi.

Il **Memory Protection Unit (MPU)**: Utilizzata per proteggere il contenuto della memoria, rende alcune regioni di memoria read-only per prevenire l'accesso da parte di applicazioni-utente.

Ha anche un **bus matrix** per trasferimento dati tra componenti e periferiche.

Questo include un AHB: Advanced High performance Bus per connessioni ad alta velocità e a più alti consumi; e un APB : Advanced Peripheral Bus interface per le periferiche (I/o, timer, interrupt, a basso consumo).

Ha anche un **debug subsystem**: When a debug event occurs: it can put the processor core in a halted state, so developers can analyse the status of the processor: such as register values and flags, at that point. Quindi sto sistema di debug può mettere il processore in stop e controllare i valori dei flag e dei registri a quel punto.

Programming model

Il programming model definisce quali sono i registri disponibili ai programmatori, mentre altri registri saranno usati per operazioni interne.

Ricordiamo che siamo in una load/store architecture.

Registri del processore = Aree di memoria temporanea all'interno del core del processore , progettate per memorizzare ed elaborare i dati in modo rapido.

Il Cortex-M4 dispone di un banco di registri composto da 16 registri (r0...r15) a 32-bit, di cui 13 sono general purpose, gli altri sono quelli speciali.

Registri General-Purpose (R0-R12)

- **R0-R12**: Sono registri generici.
 - I **Low registers (R0-R7)** possono essere acceduti da qualsiasi istruzione.
 - Gli **High registers (R8-R12)**, invece, a volte non possono essere acceduti da alcune istruzioni.

Registri Special-Purpose (R13-R15)

- **R13: Stack Pointer (SP)**: Registra l'indirizzo corrente dello stack. Viene utilizzato per salvare il contesto di un programma durante il cambio tra i task. Esistono due tipi di stack pointer: il Main SP (MSP) e il Process SP (PSP).
- **R14: Link Register (LR)**: Utilizzato per memorizzare l'indirizzo di ritorno di una sottoroutine o di una chiamata di funzione. Dopo che una funzione è terminata, il Program Counter (PC) carica il valore dal LR per ritornare al programma principale.
- **R15: Program Counter (PC)**: Contiene l'indirizzo del codice dell'istruzione corrente. Viene automaticamente incrementato di quattro ad ogni operazione (per un codice istruzione a 32-bit), ad eccezione delle operazioni di salto (branching).

Registri Speciali Aggiuntivi

Oltre ai registri R0-R15, il Cortex-M4 include altri registri speciali:

- **xPSR (Program Status Register)**: È un registro di stato del programma combinato che fornisce informazioni sull'esecuzione del programma e sui flag dell'ALU (Arithmetic Logic Unit). Combina tre sottoregistri:

- **APSR (Application Program Status Register)**: Contiene i flag N, Z, C, V, Q.
 - **N (Negative flag)**: Indica se il risultato di un'operazione è negativo.
 - **Z (Zero flag)**: Indica se il risultato di un'operazione è zero.
 - **C (Carry flag)**: Indica un riporto o un prestito in operazioni aritmetiche.
 - **V (Overflow flag)**: Indica un overflow in operazioni aritmetiche con numeri con segno.
 - **Q (Stick saturation flag)**: Indica saturazione, ad esempio in operazioni di saturating arithmetic usate nell'elaborazione audio/video per mantenere i livelli del segnale entro i limiti definiti.

- **IPSR (Interrupt Program Status Register)**: Contiene il numero dell'ISR (Interrupt Service Routine) corrente.

- **EPSR (Execution Program Status Register)**: Contiene informazioni sullo stato di esecuzione e di istruzioni interbloccate.

• **Registri di mascheramento degli interrupt:** Questi registri, come **PRIMASK**, **FAULTMASK** e **BASEPRI**, sono utilizzati per controllare il mascheramento degli interrupt e gestire le priorità.

Memory Map

La memory map descrive come viene organizzato lo spazio di indirizzamento del processore. Il processore Arm Cortex-M4 ha uno spazio di indirizzamento di 4 GB, essendo la memoria indirizzabile per byte tramite indirizzi a 32 bit.

Questo spazio di 4 GB è suddiviso in varie regioni, ognuna con un utilizzo raccomandato, che facilita la portabilità del software tra dispositivi diversi (che utilizzano lo stesso processore). Sebbene la mappa sia definita a livello architetturale, può essere anche parzialmente definita in modo flessibile dall'utente, ad eccezione di alcuni indirizzi fissi (come quelli del bus delle periferiche private interne).

Le principali regioni di memoria sono:

- Code region:** Utilizzata per memorizzare il codice del programma, ma talvolta anche per i dati. Implementata come FLASH on chip.
- Sram Region:** Utilizzata principalmente per memorizzare i dati, come heap e stack. SRAM on chip.
- Peripheral Region:** Usata per le periferiche on-chip, come quelle connesse ai bus AHB e APB.
- External RAM Region:** E' off chip, dedicata a memorie esterne come DDR. Usata per memorizzare grandi blocchi di dati, ovviamente più lenta della SRAM on chip.
- External Device Region:** Mappata a dispositivi esterni come schede SD.
- Private Peripheral Bus Region:** Fornisce accesso a risorse interne ed esterne del processore, include componenti come:
 - System Control Space che contiene il Nested Vectored Interrupt Controller.

Bit-band Operations

Le operazioni di bit banding sono una feature che consente a una singola operazione di caricamento/archiviazione (load/store) di accedere a un singolo bit in memoria.

Come funzionano:

Normalmente, senza bit-banding, per modificare un singolo bit all'interno di una parola di dati a 32 bit, è necessario eseguire una sequenza di operazioni nota come read-modify-write. Questa procedura consiste in:

1. Leggere l'intero valore a 32 bit dalla memoria.
2. Modificare il singolo bit desiderato (usando operazioni bitwise come OR o AND) mantenendo gli altri 31 inalterati.
3. Scrivere il nuovo valore a 32 bit nuovamente nella stessa locazione di memoria.

Il problema è che questa sequenza non è atomica, e si rischia race condition (ad esempio nel caso in cui occorra un interrupt dopo la lettura ma prima della scrittura).

La soluzione: Bit Band Alias

Le operazioni di bit-band risolvono questo problema mappando ogni singolo bit di una specifica regione a un indirizzo univoco "alias" di 32 bit. In questo modo con una singola istruzione hardware, l'operazione di modifica di un bit diventa atomica (per impostare un bit, si scrive semplicemente '1' al suo indirizzo alias).

Le regioni SRAM e Peripheral contengono delle aree speciali abilitate per il bit banding:

- **Bit-band region:** E' l'area di memoria fisica dove stanno i dati
- **Bit-band alias region:** E' un'area di memoria molto più grande dove ogni singolo bit della bit-region è mappato su un intero indirizzo virtuale a 32 bit. Esempio: 1 byte di memoria reale occupa 8 bit * 4 byte nella regione alias.

Ora, esiste una formula specifica che il programmatore usa per tradurre l'indirizzo di un bit nel suo corrispondente alias: $\text{AliasAddr} = \text{AliasBase} + (\text{ByteOffset} \times 32) + (\text{BitNumber} \times 4)$

Dove:

- AliasBase = indirizzo iniziale della regione alias
- ByteOffset = distanza in byte dell'indirizzo reale dall'inizio della regione bit-band
- BitNumber (0-7) : La posizione del bit che si vuole modificare.

Vantaggi del Bit-Banding:

Le operazioni sui bit sono + veloci e richiedono meno istruzioni. Inoltre l'accesso al bit tramite il suo alias è atomico, cioè indivisibile. Questo fa evitare le race conditions. Invece ad esempio se un interrupt si verificasse durante una sequenza della classica read-modify-write, la ISR potrebbe modificare lo stesso dato, causando un conflitto. Invece col bit banding ci si assicura che l'accesso al bit sia completato senza interruzioni.

Endianness

L'**endianness** è l'ordine con cui i byte di dati vengono ordinati in memoria.

Il processore Cortex-M4 supporta sia la modalità little endian che quella big endian.

Little-endian: Il byte meno significativo di una parola viene memorizzato all'indirizzo di memoria più basso.

Big-endian: Il byte meno significativo di una parola dati viene messo nella zona di memoria più alta.

Set di istruzioni

Inizialmente i processori ARM utilizzavano un set di istruzioni a 32 bit, noto semplicemente come ARM instructions, ma questo set presentava alcuni svantaggi:

- Maggiori dimensioni del programma
- Maggior consumo energetico : dovuto a necessità di spostare più informazioni da e verso la memoria.

Poi è stato introdotto il set di istruzioni **Thumb-1**:

- istruzioni a 16 bit: ogni istruzione Thumb-1 ha un'istruzione ARM a 32 bit corrispondente che esegue la stessa operazione
- **Riduce dimensioni del codice**
- Tuttavia a volte sono necessarie più istruzioni per eseguire la stessa operazione → **porta a una riduzione delle prestazioni.**

Grazie a un multiplexer interno, i processori potevano passare tra uno stato ARM e uno stato Thumb per beneficiare di entrambi.

Un'evoluzione è stato il set **Thumb-2**:

- Set di istruzioni misto: combina sia istruzioni di Thumb-1 a 16 bit sia nuove istruzioni a 32 bit
- Offre il meglio di entrambi i mondi: **densità di codice delle istruzioni a 16 bit + prestazioni delle 32 bit**, eliminando il dover passare da uno stato all'altro.

3) Embedded Program Development

Tipicamente, il flusso di generazione di un programma per un sistema embedded, specialmente per architetture come la Cortex-M4, segue una serie di passaggi definiti che trasformano codice sorgente in immagine eseguibile (pronta per essere caricata sul dispositivo).

Questo processo è anche chiamato compilazione offline, queste le sue fasi:

1. Compilazione : sorgente (es: linguaggio C) -> assembly
2. Assemblaggio: assembly -> object code (rappresentazione binaria delle istruzioni macchina)
3. Linking: Il linker prende uno o più file oggetto e li combina con le librerie necessarie, per creare un singolo file eseguibile, chiamato anche immagine del programma.
4. Download: L'immagine del programma finale viene trasferita dalla macchina host alla memoria del sistema embedded (come una flash on-chip). Da qui poi il processore fetcherà...

Program image

L'immagine del programma è un blocco di codice completamente integrato e pronto per l'esecuzione, che viene integrato nella "Code region" della memoria. Nel Cortex-M4, questa immagine contiene diverse componenti fondamentali:

- **Vector Table**: una tabella che contiene gli indirizzi di partenza delle routine di gestione delle interrupt (interrupt vectors), e il valore iniziale dello Stack Pointer (MSP) e l'indirizzo del reset vector che è il punto di partenza dall'esecuzione del programma dopo un reset.
- **C Start-up Code**: Una routine di avvio, inserita automaticamente dal compiler/linker, ha la funzione di inizializzare le variabili globali
- **Program Code**: sezione che contiene le istruzioni e i dati generati dal codice sorgente dell'applicazione scritta da noi
- **C Library Code**: Include il codice proveniente dalle librerie C standard, che viene aggiunto dal linker per risolvere chiamate a funzioni non definite nel mio codice.

Processo di inizializzazione del sistema

Al momento del reset, il processore esegue una sequenza di avvio ben definita:

1. Legge il valore del Main Stack Pointer MSP (all'indirizzo 0x00000000)
2. Legge l'indirizzo del reset vector (0x00000004)
3. Salta all'indirizzo contenuto nel reset vector, che corrisponde all'indirizzo del Reset Handler e inizia a eseguire le sue istruzioni. (0x00000008)
4. Esegue il codice di avvio (C start-up code), che prepara l'ambiente di runtime

In sintesi, la *program image* non è solo il codice dell'applicazione, ma un pacchetto completo che include la configurazione iniziale del processore e l'ambiente di runtime necessario per eseguire correttamente il programma su un sistema embedded come il Cortex-M4.

Program image → è memorizzata nella code region, implementata di solito come on chip flash memory (non volatile).

Dati del programma → nella SRAM (che contiene Stack, Heap, variabili globali/locali...).

Localizzazione Dati nella SRAM

La regione di memoria SRAM è usata per memorizzare dati che possono cambiare durante l'esecuzione del programma (è volatile, ma veloce). È suddivisa in 3 aree:

- **Static Data**: Questa area contiene variabili globali e statiche, inizializzate e non.
- **Stack**: È un'area che cresce e si restringe dinamicamente. Contiene variabili locali alle funzioni (questo spazio viene liberato alla fine della funzione). Lo stack viene anche usato per il passaggio dei parametri durante le chiamate a funzione, il salvataggio dell'indirizzo di ritorno da una funzione, il salvataggio del contesto del processore (registri) durante la gestione di eccezioni/interrupt.
- **Heap**: È un'area di memoria utilizzata per l'allocazione dinamica tramite funzioni come malloc() o alloc(). L'uso dell'heap è generalmente sconsigliato nei sistemi embedded.

perchè può portare a una gestione della memoria complessa /problemi di frammentazione.

Di quali memorie necessita un programma?

Dipende dalle esigenze.

La **memoria non volatile** conserva le informazioni anche quando dispositivo è spento, è quindi usata per memorizzare tutto ciò che non cambia durante l'esecuzione: istruzioni del programma, costanti, valori di inizializzazione..

La **memoria volatile**, solitamente SRAM on-chip, è necessaria per tutti quei dati che possono cambiare durante esecuzione: variabili globali/locali, stack, heap, calcoli intermedi...

Inoltre la memoria viene gestita diversamente anche a seconda di quanto a lungo i dati devono persistere:

- Allocazione statica : i dati esistono dall'inizio alla fine del programma. Ogni variabile ha una posizione fissa e lo spazio non viene riusato.
- Allocazione automatica : I dati esistono solo dall'inizio alla fine di una funzione, lo spazio viene riusato automaticamente quando la funzione termina.
- Allocazione dinamica: L'allocazione e deallocazione sono esplicite.

In sintesi, **la Flash ROM contiene l'intera program image**, che include il codice e i dati "sorgente" per l'inizializzazione, mentre **la SRAM ospita i dati "vivi"** su cui il programma opera attivamente.

Image sections:

Il linker produce un eseguibile organizzato in sezioni

-.**text**: risiede in Flash, contiene codice eseguibile, istruzioni

-.**const**: risiede in Flash, dati di sola lettura

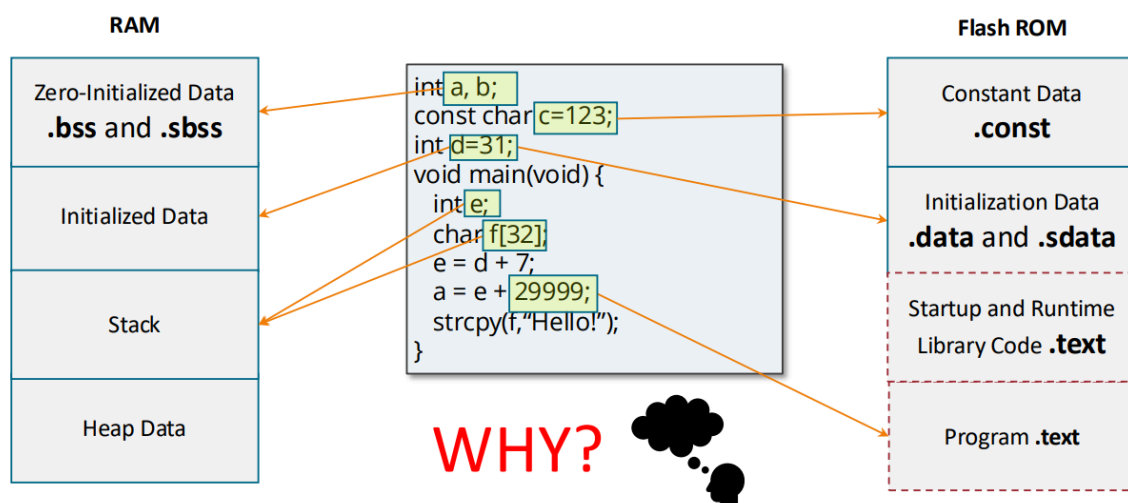
-.**data**: risiede in SRAM, variabili globali e statiche inzializzate. I valori iniziali sono slavati in Flash ma all'inizio dell'esecuzione questi valori vengono copiati da Flash a SRAM per poter essere modificati.

-.**bss**: Variabili globali/statiche non inzializzate (o inzializzate a zero), in SRAM.

-.**Stack**: Contiene variabili locali di funzione, cresce verso il basso.

-.**Heap**: Cresce verso l'alto e gestisce memotia allocata dinamicamente

Program Memory Use



Nota: Il valore numerico 29999 è memorizzato nella sezione .text perchè è classificato come “operando costante”, infatti non viene mai modificato e quindi deve risiedere nella memoria non volatile Flash / ROM. Inoltre quel 29999 è parte integrante dell’istruzione stessa (in assembly è un immediato, per questo va in .text che è deputata a contenere le istruzioni).

C runtime startup module:

E’ una componente software inserita automaticamente dal compilatore o dal linker nell’immagine del programma. Ha la funzione di preparare l’ambiente necessario affinché un programma C venga eseguito correttamente su un MCU, specialmente dopo il reset.

Caricamento dell’immagine del programma:

L’immagine del programma (eseguibile), viene memorizzata nella memoria non volatile (Flash on CHip), all’interno della Code region.

Procedura di avvio(booting)

Dopo l’accensione di un sistema embedded, il processore inizia a recuperare il codice da un indirizzo di memoria predefinito e cablato.

Questo codice è chiamato reset vector. Solitamente il reset vector è un’istruzione di salto che indirizza il processore verso il codice di inizializzazione vero e proprio (bootstrap code).

La ragione di questo salto da un’altra parte della memoria è mantenere il vettore di reset il più piccolo possibile.

La procedura di avvio coinvolge principalmente 2 registri:

- Instruction Pointer(IP): Punta alla prossima istruzione da eseguire (codice in .text)
- Stack Pointer (SP) : Punta al prossimo indirizzo libero nello stack. E’ molto importante che l’SP venga impostato correttamente all’avvio, poichè lo stack viene creato in uno spazio della RAM.

Ecco i passaggi:

1. Inizialmente l’IP è cablato per eseguire la prima istruzione (ossia il reset vector)
2. Reset vector effettua un salto alla prima istruzione della sezione .text situata nella ROM
3. Il codice di bootstrap copia la sezione .data dall’immagine di avvio (in ROM) alla RAM.
4. Viene riservato spazio in RAM per la sezione .bss, che contiene le variabili non inizializzate, e riempie quest’area con tutti zeri.
5. Viene riservato spazio per lo stack in RAM, e il registro SP viene impostato.

Alla fine, la CPU continua con l’esecuzione della .text section.

Linker:

Il linker prende in input i file oggetto e le librerie e combina in un unico file eseguibile (program image). Il linker svolge due compiti principali:

1. Unisce le sezioni: Combina le sezioni di codice e dati provenienti dai vari file oggetti e li organizza in segmenti.
2. Alloca memoria: Assegna questi segmenti agli indirizzi di memoria specifici del sistema target.

Il **Linker Command File** è un file di testo che contiene le direttive per guidare il processo di linking. Attraverso questo file, lo sviluppatore “istruisce” il linker su come mappare l’immagine del programma sulla memoria fisica del sistema embedded.

Le due direttive più importanti supportate dalla maggior parte dei linker sono MEMORY e SECTION.

La direttiva **MEMORY** serve a descrivere la mappa di memoria fisica del sistema target. Per un sistema embedded, è fondamentale che lo sviluppatore conosca l'organizzazione della memoria (ROM, FLASH, RAM), i loro indirizzi di inizio e le loro dimensioni.

Esempio: MEMORY {

ROM: origin = 0x0000h, length = 0x0020h

FLASH: origin = 0x0040h, length = 0x1000h

RAM: origin = 0x1000h, length = 0x10000h

}

La direttiva **SECTION** specifica come il linker deve combinare le sezioni di input (dai file oggetto) in sezioni di output e dove queste ultime devono essere collocate nella memoria definita dalla direttiva MEMORY.

Le sezioni di un programma includono:

- .text: Contiene codice eseguibile e costanti
- .data: Variabili globali e statiche inizializzate
- .bss: variabili globali e statiche non inizializzate

Poi ci possono essere anche sezioni personalizzate.

4) PROCESSORS AND I/O

Come già detto la MCU è un sistema complesso che integra una CPU, memoria e varie periferiche I/O su un singolo chip. La CPU comunica con queste periferiche (dispositivi I/O) leggendo e scrivendo su speciali locazioni di memoria chiamate **device registers**.

Quindi ogni dispositivo I/O ha tipicamente diversi registri:

- **Data registers**: Contengono i dati effettivi da trasferire. Possono essere di sola lettura (per l'input) o di sola scrittura (per l'output).
- **Status Registers**: Forniscono informazioni sullo stato operativo del dispositivo, ad esempio, se un'operazione è stata completata o se il dispositivo è pronto per una nuova operazione, questi possono essere solo read-only.

Esistono due modi principali per accedere ai device registers:

1. Istruzioni I/O Speciali: Alcune architetture, come Intel x86, hanno istruzioni dedicate (ad es. in e out) che operano in uno spazio di indirizzi separati per l'I/O.
2. Memory-Mapped I/O: Questo è l'approccio più comune nei moderni microcontrollori come l'ARM Cortex-M4. I registri di ogni dispositivo I/O sono mappati a indirizzi specifici all'interno della mappa di memoria principale del processore. Per comunicare con il dispositivo, il programma utilizza le normali istruzioni load e store della memoria normale, come se stesse accedendo a una variabile in RAM.

Ad esempio, in un sistema ARM, si definisce l'indirizzo del dispositivo e poi si usano puntatori per leggere o scrivere su quel registro.

Scrivere su dispositivi I/O in C

Il C è ideale per questo compito perchè permette la manipolazione diretta degli indirizzi di memoria tramite l'uso di puntatori.

Il primo passo è definire gli indirizzi dei registri del dispositivo con cui si vuole comunicare.

Questo viene fatto usando la direttiva #define per creare delle costanti simboliche, rendendo il codice più leggibile. Ad esempio:

```
#define OUT_CHAR 0x1000 // Indirizzo del registro dati del dispositivo di output
```

```
#define OUT_STATUS 0x1001 // Indirizzo del registro di stato del dispositivo di output
```

Si possono poi creare delle funzioni di utility che prendono un indirizzo e un valore, per poi leggere o scrivere su quella specifica locazione. Praticamente servono ad astrarre l'operazione di dereferenziazione del puntatore.

```
// Funzione per scrivere un valore in una locazione di memoria arbitraria
```

```
void write(char *location, char newval) {
```

```

    (*location) = newval; // Dereferenzia il puntatore e assegna il nuovo valore
}
// Funzione per leggere un valore da una locazione di memoria arbitraria
int read(char *location) {
    return *location;
}

```

Una volta definite le costanti e le funzioni, scrivere un valore (ad esempio, il numero 8) in un registro del dispositivo diventa molto semplice:

```

dev_status = read(DEV1);
// Scrive il valore 8 nel registro del dispositivo all'indirizzo 0x1000
write(OUT_CHAR, 8);

```

Busy-Wait I/O:

Il Busy-Wait I/O, spesso chiamato **polling**, è la tecnica più semplice per sincronizzare la comunicazione tra la CPU e un dispositivo periferico (I/O). Dato che i dispositivi I/O sono tipicamente molto più lenti della CPU, è necessario un meccanismo per assicurarsi che un'operazione sia completata prima di iniziarne una nuova.

Il principio del busy-wait consiste nel far sì che la **CPU controlli attivamente e ripetutamente lo status register di un dispositivo** per determinare se è pronto per una nuova operazione. In pratica la CPU entra in un ciclo di attesa (busy loop) dal quale esce solo quando lo stato del dispositivo cambia.

Come funziona a livello pratico?

Per comunicare con un dispositivo, la CPU legge e scrive su appositi registri mappati in memoria (memory-mapped I/O). I registri tipici sono:

- **Data register:** Dove vengono scritti i dati da inviare o da cui vengono letti i dati ricevuti
- **Status Register:** Indica lo stato operativo del dispositivo (busy(1) oppure completato(0)).

Esempio: Scrittura di una stringa di caratteri:

Immaginiamo di voler inviare una stringa di caratteri a un dispositivo di output, come una porta seriale. Il dispositivo ha due registri:

1. **OUT_CHAR:** Il registro dati dove scriviamo il carattere da inviare.
2. **OUT_STATUS:** Il registro di stato che vale **1** se il dispositivo è occupato a inviare il carattere precedente e **0** quando è libero e pronto a riceverne un altro.

Il codice C per inviare la stringa "Hello, world." usando il busy-wait sarebbe simile a questo:

```

#define OUT_CHAR 0x1000 // Indirizzo del registro dati del dispositivo
#define OUT_STATUS 0x1001 // Indirizzo del registro di stato del dispositivo
char *mystring = "Hello, world.";
char *current_char = mystring;
// Itera su ogni carattere della stringa finché non incontra il terminatore nullo '\0'
while (*current_char != '\0'){
    // 1. Scrive il carattere corrente nel registro dati del dispositivo
    write(OUT_CHAR, *current_char);
    // 2. BUSY-WAIT: Entra in un ciclo di attesa attiva (polling)
    // Continua a leggere il registro di stato finché il suo valore non diventa 0
    while (read(OUT_STATUS) != 0);
    // 3. Una volta che il dispositivo è pronto, passa al carattere successivo
    current_char++;
}

```

In questo esempio, il ciclo `while (read(OUT_STATUS) != 0);` è il cuore del busy-wait. La CPU non fa altro che eseguire ripetutamente l'istruzione di lettura dello stato, sprecando cicli di calcolo finché l'operazione I/O non è terminata.

Esempio 2: Lettura e Scrittura Continua

Un altro esempio comune è un programma che legge un carattere da un dispositivo di input e lo invia a un dispositivo di output in un ciclo infinito. Anche in questo caso, si usano cicli di busy-wait per sincronizzare sia la lettura che la scrittura.

```
#define IN_DATA    0x1000
#define IN_STATUS  0x1001
#define OUT_DATA   0x1100
#define OUT_STATUS 0x1101
while (TRUE) {
    // ATTESA PER LA LETTURA
    // Aspetta finché il dispositivo di input non ha un nuovo carattere (status == 1)
    while (read(IN_STATUS) == 0);
    achar = (char)read(IN_DATA); // Legge il carattere
    // PREPARAZIONE PER LA SCRITTURA
    write(OUT_DATA, achar);      // Scrive il carattere sul dispositivo di output
    write(OUT_STATUS, 1);        // Imposta lo stato per iniziare la scrittura
    // ATTESA PER LA SCRITTURA
    // Aspetta finché il dispositivo di output non ha finito (status == 0)
    while (read(OUT_STATUS) != 0);
}
```

All'inizio la CPU continua a interrogare il dispositivo di input, finché risponde 0, aspetta. Appena risponde 1 (es: l'utente ha premuto un tasto) il ciclo si rompe. Dopo la CPU legge il dato, e lo scrive su OUT_DATA, e imposta esplicitamente OUT_STATUS a 1(occupato). Poi la CPU aspetta che il dispositivo di output completi il lavoro prima di tornare a leggere il prossimo. Per farla breve qua ci sono due cicli di busy-wait: uno per attendere l'arrivo di un dato in input e uno per attendere il completamento della scrittura in output.

Vantaggi e svantaggi Busy wait I/O:

- **Vantaggio - Semplicità** : E' la tecnica più semplice, la logica del programma è facile da comprendere
- **Svantaggio - Inefficienza**: Il busy wait fa sprecare alla CPU una grande quantità di tempo e cicli in esecuzione, senza poter svolgere alcun altro lavoro utile in parallelo con l'operazione di I/O.

INTERRUPTS

Il metodo degli interrupts risolve il problema dell'inefficienza del busy wait invertendo il meccanismo di controllo: invece che la CPU che chiede al dispositivo se è pronto, **è il dispositivo a segnalare alla CPU quando ha bisogno di attenzione.**

Questo permette alla CPU di eseguire altre operazioni utili mentre il dispositivo I/O lavora in parallelo.

Il processo di un'interruzione è controllato da diversi segnali:

1. **Interrupt Request (IRQ)** : Quando il dispositivo di I/O necessita di un servizio da parte della CPU, esso manda un segnale di richiesta di interrupt.

2. **Verifica e riconoscimento:** La CPU verifica la linea di richiesta di interrupt (IRQ) ad ogni istruzione. Quando la CPU è pronta a gestire la richiesta, asserisce il segnale di **interrupt acknowledge signal**.
3. **Decisione del Dispositivo:** La logica del dispositivo di I/O decide autonomamente quando interrompere (es: alla pressione del bottone).

Quando un interrupt viene riconosciuto, il flusso di controllo della CPU cambia:

la CPU non esegue l'istruzione puntata dal Program Counter (PC). Invece viene salvato il valore del PC corrente su uno stack, in modo da poter ritornare al programma principale (foreground program) in un secondo momento. La CPU poi imposta il PC all'inizio della routine di gestione dell'interrupt del dispositivo, nota come Interrupt Handler o **Interrupt Service Routine (ISR)**.

A questo punto avviene l'esecuzione dell'ISR in cui l'handler si occupa del dispositivo, ad esempio leggendo i dati appena pronti o scrivendo i dati successivi.

Dopo che l'ISR ha completato il suo compito, l'istruzione di ritorno dall'interrupt ripristina lo stato salvato automaticamente, permettendo al programma in foreground di riprendere la sua esecuzione regolare.

Nota: Nei sistemi embedded, è comune che il software sia organizzato come un sistema **foreground/background**, dove la logica principale risiede nel *main code* (foreground) e tutte le operazioni di I/O sono gestite dalle ISR (background).

Esempio 1: Copiare caratteri da input ad output con degli interrupt: versione base

```
INTERRUPT HANDLERS
/* get a character and put in global (called when IN_STATUS is 1) */
void input_handler() {
    achar = read(IN_DATA); /* get character */
    gotchar = TRUE;        /* signal to main program */
    write(IN_STATUS,0);    /* reset status to initiate next transfer */
}

/* react to character being sent (called when OUT_STATUS is 0) */
void output_handler() {
    /* don't have to do anything */
}
```

```
main() {
    while (TRUE) { /* read then write forever */
        if (gotchar){ /* write a character */
            write(OUT_DATA,achar); /* put character in device */
            write(OUT_STATUS,1);   /* set status to initiate write */
            gotchar = FALSE;       /* reset flag */
        }
    }
}
```

Osservazioni primo esempio: Qui il main ora è più semplice rispetto a quello con polling/busy wait. Tuttavia questo programma non va bene, perchè still non permette al foreground program di fare lavoro utile, infatti è ancora in un'attesa attiva di ricevere un carattere.

Esempio 2: Copiare caratteri da input a output con Interrupts e buffer circolare

Usiamo un buffer circolare per tenere i caratteri: ha un puntatore **head** alla posizione del primo carattere inserito (più vecchio) e un puntatore **tail** alla prossima posizione libera. Quando gli indici arrivano alla fine dell'array : tornano a 0. L'idea di questo metodo è disaccoppiare la velocità di arrivo dei dati di input dalla velocità di trasmissione in output.

Funzioni di supporto :

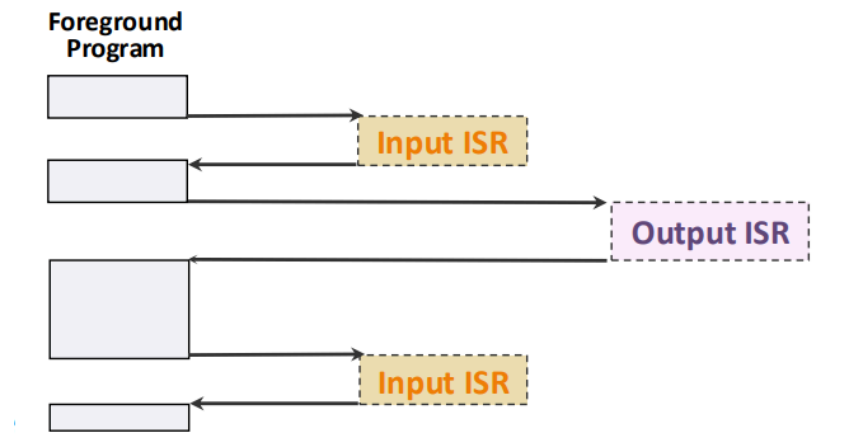
- **add_char(char)** : Inserisce un carattere alla posizione tail e incrementa l'indice
- **remove_char()** : Preleva un carattere dalla posizione head (FIFO) e incrementa l'indice.
- **nchars()** : Restituisce il numero di caratteri attualmente nel buffer.

Funzione **input_handler()**: Viene chiamata dall'hardware quando un nuovo dato è disponibile nel registro di input (IN_STATUS è 1). Questa controlla se il buffer è pieno, se c'è spazio leggere il carattere da IN_DATA e lo salva nel buffer usando add_char(); infine azzerava IN_STATUS.

Dopo di che il codice controlla se nchars()==1: Se il numero di caratteri è 1, prima di questo inserimento il buffer era vuoto, dunque il dispositivo di output era inattivo, quindi scrivo il carattere su OUT_DATA e attivo la trasmissione con OUT_STATUS = 1.

Funzione **output_handler()** : Viene chiamata dall'hardware quando il dispositivo di output ha finito di trasmettere (OUT_STATUS torna a zero).

Programma principale (main) : mentre tutto accade, il main non si occupa minimamente dell'I/O e si fa i cazzacci suoi moltiplicando matrici ad esempio. Ogni tanto viene interrotto e dopo che son finiti gli interrupt handler il foreground program continua nel suo corso regolare.



Bugs nei codici con Interrupts:

Gli errori possono essere molto difficili da trovare quando gli interrupt handlers sono buggati a causa della **concorrenza**. Un errore fondamentale riguarda la gestione dei registri della CPU. Quando un'interruzione prende il controllo, l'ISR utilizza i registri della CPU per eseguire le proprie operazioni. Se l'ISR modifica un registro che il programma principale stava utilizzando, senza ripristinarlo prima di uscire, il programma in foreground si troverà con dei valori corrotti, e cambiamenti misteriosi nelle variabili del programma principale, rendendo il debug molto difficile.

Come vengono implementati gli Interrupts:

La CPU checka la linea di interrupt request (IRQ) a ogni istruzione. Se trova una richiesta di interrupt la CPU deve prima gestire l'istruzione che sta eseguendo: se l'istruzione è breve allora la completa, se è complessa (come Load o Store) la abbandona, e verrà ripresa al termine degli interrupt. Dopo di che la CPU salva il contesto corrente nello stack per poter ritornarci. A questo punto la CPU non fetcha l'istruzione al PC, ma imposta il PC all'inizio dell'interrupt handler.

Gli indirizzi degli interrupt handlers sono memorizzati in una tabella.

Il dispositivo che richiede l'interrupt infatti invia un "numero di vettore", oppure la CPU lo determina tramite il controllore **NVIC (Nested Vectored Interrupt Controller)**. La CPU usa questo numero come indice per accedere alla **Interrupt Vector Table**, e qui legge l'indirizzo della funzione ISR/Handler specifica per quell'interrupt.

Il processore Cortex-M4 include componenti hardware specifici per la gestione degli interrupt:

- **Controller degli interrupt:** Le periferiche utilizzano gli interrupt per notificare la CPU riguardo agli eventi. L'architettura MCU include un Interrupt Controller.
- **Nested Vectored Interrupt Controller (NVIC):** Questo componente gestisce automaticamente gli interrupt annidati (nested interrupts) e confronta le priorità tra le richieste di interrupt e il livello di priorità corrente. L'NVIC fa parte del Private Peripheral Bus (PPB).
- **Wake-up Interrupt Controller (WIC):** Nelle applicazioni a basso consumo, il microcontroller può entrare in modalità sleep. Quando viene rilevata una richiesta di interrupt, il WIC può informare l'unità di gestione dell'alimentazione di riattivare il sistema.

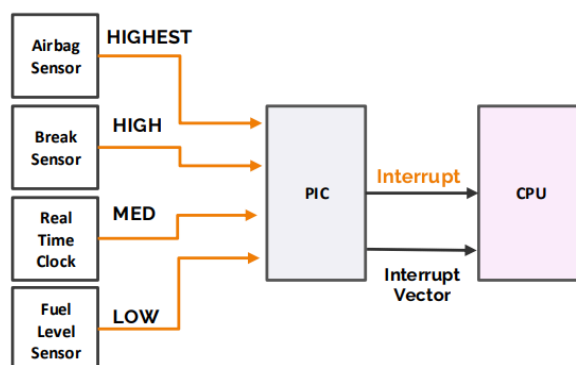
Priorità e Vettori:

La maggior parte dei sistemi embedded presenta più di un dispositivo I/O, il che richiede un sistema per gestire interruzioni multiple.

1. Priorità degli interrupt

Quando più dispositivi richiedono l'attenzione della CPU contemporaneamente, è necessario un meccanismo per decidere quale richiesta servire prima. In pratica il sistema deve riconoscere alcuni interrupt come più importanti di altri.

Per gestire queste sorgenti multiple, si utilizza un **Programmable Interrupt Controller (PIC)**, il cui ruolo è quello di dare la priorità alle diverse sorgenti di interrupt in modo che, in qualsiasi momento, l'interrupt con priorità più alta venga presentato al core della CPU per l'elaborazione.



Inoltre il meccanismo di priorità consente a un interrupt con priorità più alta di interrompere (**preempt**) l'elaborazione di un interrupt con priorità più bassa.

Un dispositivo sa che la sua richiesta di interruzione è stata accettata quando riceve un **interrupt acknowledge** che gli mostra anche il suo numero di priorità.

Interrupt Masking: Questo meccanismo invece assicura che un interrupt a priorità più bassa non si verifichi mentre un interrupt a priorità più alta è in fase di gestione.

Quando un interrupt viene riconosciuto, la CPU memorizza il **livello di priorità** di quell' interrupt in un registro interno.

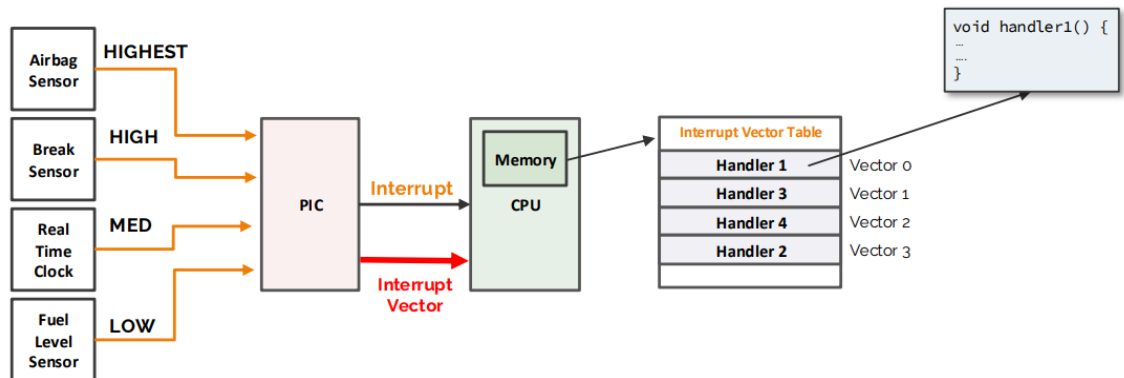
Quando viene ricevuta una richiesta di interrupt successiva, la sua priorità viene confrontata con il livello di priorità corrente. L'interrupt successivo viene riconosciuto **solo se ha una priorità superiore**.

L'interrupt con la priorità più alta è anche detto **Nonmaskable Interrupt (NMI)**, questo non può essere mascherato ed è generalmente riservato per eventi critici (ad esempio guasti di alimentazione) al fine di salvare lo stato essenziale in memoria non volatile o spegnere i dispositivi I/O in sicurezza.

2. Vettori di interrupt e tabella dei vettori

Una volta che un interrupt è stato riconosciuto, la CPU deve sapere quale routine specifica eseguire per servire il dispositivo: l'**Interrupt Service Routine**.

Interrupt Vector Number: Inviato dal dispositivo dopo che è stato riconosciuto. Questo numero è un **indice** per la **Tabella dei Vettori di Interrupt**, che è memorizzata in memoria (è inclusa nell'immagine del programma e si trova all'inizio della Code Region). Nella posizione sulla matrice fornita dall'Interrupt Vector Number, si trova l'**Interrupt Vector**, che è l'indirizzo di memoria della routine di gestione dell'interrupt specifica per quel dispositivo.



Overhead negli interrupt:

Si riferisce al tempo aggiuntivo richiesto dalla CPU per sospendere l'esecuzione del programma principale, eseguire l'ISR, e ritornare al punto di interruzione.

L'interrupt provoca un cambiamento del PC, che comporta una penalità : **branch penalty**.

Inoltre l'hardware può salvare automaticamente alcuni registri della CPU sullo stack, e anche questo richiede cicli extra. Inoltre la CPU deve dedicare cicli extra anche per riconoscere l'interrupt e ottenere il vettore dal dispositivo che lo ha interrotto.

Infine l'istruzione di ritorno dall'interrupt ripristina lo stato salvato automaticamente, il che comporta nuovamente una **penalità di salto**.

Nei sistemi in tempo reale (come quelli basati su Cortex-M4), l'**Enhanced Determinism** è una caratteristica cruciale, assicurando che i compiti critici e le routine di interrupt possano essere serviti rapidamente e in un numero noto di cicli, mitigando l'impatto dell'overhead.

Il tempo richiesto dall'hardware per rispondere all'interrupt, ottenere il vettore, salvare lo stato eccetera non può essere modificato dal programmatore.

5) Introduction to TI MSP432 Launchpad

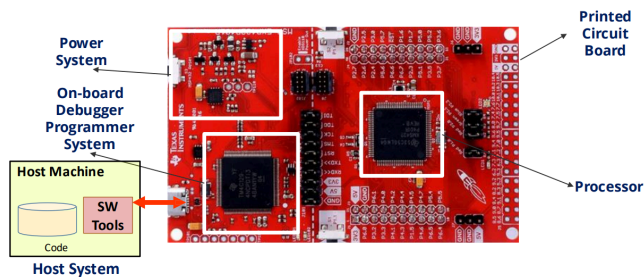
Le componenti principali della piattaforma di sviluppo sono:

Sistema host : Il computer su cui programmatore scrive codice. include:

- Cross-compiler: Un compilatore che gira sull'host ma genera codice eseguibile per un'architettura di processore diversa, quella del sistema target.
- Linker
- Debugger a livello di codice sorgente

Sistema target (debugger, processore, sensori)

Connessione host <-> target : Serve per il downloading delle immagini di programma dall'host al target. Serve anche per trasmettere informazioni di debug tra il debugger dell'host e l'agente di debug sul target.



Main features della scheda

La piattaforma ha 100 pin.

MSP = Mixed Signal Processor (analogico + digitale)

Ha 256 KBytes di on-chip-Flash per la memoria del codice.

Ha 64KB di on-chip-SRAM per i dati.

Ha memory-mapped I/O.

6) Bare Metal Programming

Input/output:

GPIO = General Purpose input output.

Pin = Connessione fisica al microcontroller

Port = Combinazioni di pins.

I pin possono avere varie funzioni, e con il registro appropriato **SEL** si seleziona quale funzione si vuole attivare : Esempio:

```
// SEL1=0 ; SEL0=0 --> GPIO normale
// SEL1=0; SEL0=1 --> Funzione primaria(Timer_A)
// SEL1=1; SEL0=0 --> funzione secondaria
// SEL1=1; SEL0=1 -->funzione terziaria
//esempio per attivare funzione terziaria su pin P2.4:
P2->SEL0 |= BIT4;
P2->SEL1 &= ~BIT4;
```

Direzione: Si Con registro **DIR**:

- 1: Output
- 0: Input

Valore logico di corrente: Con registro **OUT**, se è a 1 allora l'output viene settato a HIGH per mandare corrente.

Nota che BIT0 è una costante definita in msp.h e sarebbe 1 in binario: 0000 0001, BIT1 è 2 in binario: 0000 0010, e così via....

Ci sono 2 tipi di LED:

LED Rosso : collegato a P1.0

LED RGB: collegato a:

- P2.0 → rosso
- P2.1 → verde
- P2.2 → blu

Pulsanti:

S1 → collegato a P1.1

S2 → collegato a P1.4

Uso del pulsante user (P1.1):

Prima di tutto si setta il pin1 della porta 1 come input:

```
P1-> DIR &= ~BIT1; //P1.1 come input
```

Ora bisogna sapere che un pin di input è come un filo collegato a un'antenna, e se non è collegato a nulla può "sentire" segnali casuali (rumore). Quindi legge un valore logico che fluttua tra 0 e 1, è imprevedibile.

Per risolvere si collega il pin a una tensione stabile (3,3 V o GND) attraverso una resistenza:

```
P1->REN |= BIT1; //abilita resistenza
```

Questa resistenza tira su il segnale (pullup) o tira giù (pull-down), così anche quando il pulsante non è premuto il pin ha uno stato logico definito. REN= Resistor Enable Register. Nota finora, ha solo collegato la resistenza al pin.

La direzione (pull-up o pull-down) dipende dal bit corrispondente in P1->OUT, se p1out = 1 allora la resistenza è collegata a 3.3 V e quindi è un pull-up; se p1out = 0 invece è pull down.

```
P1->OUT|= BIT1; //OUT= 1, pull-up abilitato
```

In conclusione con pull-up : pulsante premuto = pin va a GND e legge BASSO.

Con pull-down = Pulsante premuto = pin va all'alimentazione VCC e legge ALTO.

Registri porta 1 comunemente usati:

Address	Name	Description	Type	Reset Value
0x4000 4C00	P1IN	Port1 Input Register	R	0b00000000
0x4000 4C02	P1OUT	Port1 Output Register	R/W	0b00000000
0x4000 4C04	P1DIR	Port1Direction Register	R/W	0b00000000
0x4000 4C06	P1REN	Port1 Resistor Enable Register	R/W	0b00000000
0x4000 4C08	P1DS	Port1 Drive Strength Register	R/W	0b00000000
0x4000 4C0A	P1SEL0	Port1Select 0 Register	R/W	0b00000000
0x4000 4C0C	P1SEL1	Port1Select 1 Register	R/W	0b00000000

7) Interrupt lab

Hardware triggered interrupts: Sono routine software asincrone innescate da un segnale hardware proveniente da una periferica interna (come un timer) o da un dispositivo esterno. Sono asincroni pk possono verificarsi in qualsiasi momento durante l'esecuzione del programma principale (a meno che non siano stati disabilitati /mascherati).Non sono legati a una specifica riga di codice.

Software triggered interrupts:

Sono interrupt che risultano dall'esecuzione di specifiche istruzioni. Spesso vengono chiamati eccezioni o faults. Sono sincroni, ossia accadono in un punto prevedibile del flusso del programma.

NVIC: Gestisce gli interrupt esterni simultanei in base alla priorità : smaller number -> higher priority. Nota che le priorità di alcuni interrupt sono fissate : ad esempio il Reset ha la priorità + alta. Invece altre priorità si possono configurare , e si possono pure mascherare alcuni interrupt (altri invece sono non maskable).

Pending Interrupt = Un interrupt che è stato richiesto e detectato ma non è ancora stato servito (magari perchè si sta servendo uno con priorità + alta).

Nested interrupt: E' una situa in cui una chiamata a una routine di interruzione avviene all'interno di un'altra interruzione che è già attiva (ad esempio un'interruzione a priorità + alta può prelatzionare l'elaborazione di una a priorità + bassa.)

Ecco in pratica come si schedulano gli interrupt:

- Simultaneous interrupt requests?
 - High priority interrupt is serviced first
- New interrupt requested while a handler is executing?
 - New priority higher than current priority?
 - *New interrupt handler preempts current interrupt handler.*
 - New priority lower than or equal to current priority?
 - *New interrupt held in pending state .*
 - *Current handler continues and completes execution.*
 - *Previous priority level restored.*
 - *New interrupt handled if priority level allows.*

La CPU checka se ci sono interrupt pendenti all'inizio di ogni istruzione, e risponde a quello con priorità maggiore. Il dispositivo scelto riceve un ACK e manda il suo vector number. La CPU riceve sto numero e lo usa come indice dell' interrupt vector table -> qui trova l'indirizzo dell' interrupt handler da eseguire.

```
//Configura interrupt su fronte di discesa (pressione del tasto)
// Ogni porta GPIO del MSP432 ha 3 registri principali per gestire interrupt
// PxIES: Interrupt edge select. Decide su quale fronte(raising/falling)
// scatta l'interrupt. Quindi il registro IES decide quando deve scattare l'interrupt.
// -Se IES = 1 --> scatta quando il segnale scende (da HIGH a LOW)
// -Se IES = 0 --> scatta quando il segnale sale (da LOW a HIGH)
// Quindi se metto pullup interno sul pulsante è giusto scegliere fronte di discesa

P1->IES |= (BIT1 | BIT4); //Fa scattare l'interrupt quando il segnale sui pin P1.1 e P1.4 passa da 1 a 0

// PxIFG: Interrupt flag. Questo registro memorizza un flag(bit) per ciascun pin
// che dice "E' successo un evento di interrupt su questo pin".
// Quando il pin rileva la transizione configurata (es. fronte di discesa), il bit IFG
// si mette automaticamente a 1. Dentro la ISR devo sempre azzerarlo, altrimenti l'interrupt si ripete subito

P1->IFG &= ~(BIT1 | BIT4); //pulsici flag, così non parte subito con un interrupt falso

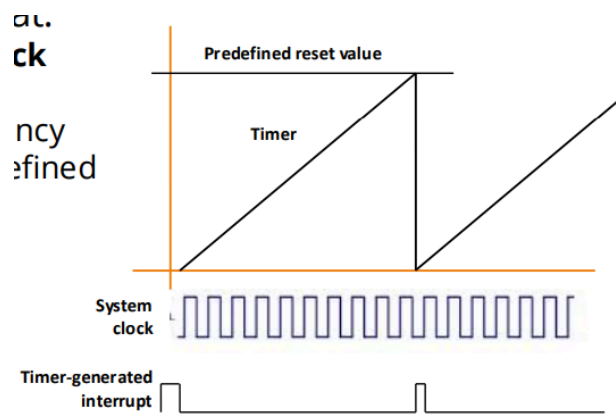
// PxIE: Interrupt Enable. Questo registro abilita/disabilita gli interrupt sui singoli pin
// -Bit = 1 --> interrupt abilitato
// -Bit = 0 --> interrupt disabilitato

P1 ->IE |= (BIT1 | BIT4); //abilita interrupt per P1.1 e P1.4

// enable_irq: è una funzione del compilatore, serve ad abilitare gli interrupt globalmente nel processore
__enable_irq();
// L'NVIC decide quale interrupt abilitare /disabilitare e in che ordine vengono gestiti.
// Ogni possibile interrupt(timer, porte, ADC, UART, ecc..) ha un suo numero identificativo chiamato IRQ number
// ISER: sta per Interrupt Set-Enable Register. Serve per abilitare gli interrupt.
// Ce ne sono più di uno (ISER[0], ISER[1], ecc...) perché ci possono essere molti interrupt
// Quindi con ISER[1] stiamo scrivendo nel secondo registro ISER (e ogni registro gestisce a sua volta 32 interrupt, quindi iser[1] è per gli irq dal 32 al 63).
// PORT1_IRQn: è il numero identificativo associato alla porta 1 del MSP, ossia una costante definita in msp.h
// &31: Prende solo i 5 bit meno significativi del numero, per capire quale bit del registro ISER accendere (è come fare %32)
// 1<<...: Questo serve a creare un numero binario con solo quel bit impostato a 1 per abilitare
// l'interrupt corrispondente
NVIC->ISER[1] = 1 << ((PORT1_IRQn) & 31); //così accendo il bit 3 di iser[1]
```

8)Timer and Clocks lab

Un **timer hardware** è un contatore digitale che conta eventi regolari usando un **clock a frequenza fissa**. Il timer resetta sé stesso quando raggiunge zero, oppure un valore predefinito, e genera un interrupt quando si resetta.



Un **timer software** invece è simile a quello hardware (in realtà si basa su timer hardware), e può avere anche multiple istanze di timer fisici, ma fornisce minor precisione.

Componenti di un timer standard:

Il timer ha un **prescaler**, che prende il clock sorgente in input, divide la frequenza di questo clock per un valore predefinito (4, 8 o 16....) e manda in output la frequenza divisa agli altri componenti. Poi c'è il **timer register che viene incrementato o decrementato a una frequenza fissa**, è guidato dall'output del prescaler.

Tipicamente un timer ha 3 modalità :

1. Compare mode
2. Capture mode
3. PWM mode

Compare mode:

C'è un **compare register** che contiene un valore, il quale viene confrontato periodicamente con il valore del timer register: Se i due valori sono uguali : si genera l'interrupt.

Capture mode:

In questa modalità si carica, quando succede un certo evento, il valore corrente del timer register nel **capture register**, e se voglio faccio anche scattare un interrupt. Opzionalmente si può usare anche un secondo prescaler applicato alla frequenza generata dall'evento, il registro di cattura di base prende uno snapshot del momento.

PWM mode:

Usa la larghezza di un impulso per modulare l'ampiezza di un segnale, gioca variando il duty cycle (rapporto tra durata del segnale a 1 e durata del segnale a 0 in un periodo). Si usa per controllare quanta potenza dare ai dispositivi elettronici come i motori.

Watchdog timer:

È un modulo dell'msp progettato per eseguire un riavvio controllato del sistema nel caso in cui si verifichi un problema software. Praticamente il timer conta alla rovescia partendo da un intervallo di tempo selezionato, se il software non interviene prima che il timer scada il Watchdog genera un reset del sistema. In un funzionamento normale il software dovrebbe "resettare" periodicamente il conteggio del Watchdog per impedirgli di scadere, segnalando così che il sistema sta funzionando correttamente.

Clock:

Il sistema di clock del msp432 è flessibile, permette di scegliere tra diverse sorgenti per bilanciare la precisione temporale e il consumo energetico.

Sorgenti di Clock:

Il microprocessore offre sia sorgenti esterne (+ precise) che interne (sempre disponibili).

Segnali di Clock:

Il modulo di clock mappa le sorgenti a cinque segnali principali che alimentano le varie parti del dispositivo (ad esempio SMCLK).

I registri del sistema di clock sono protetti da scrittura accidentale, per modificarli è necessario sbloccare il modulo scrivendo la chiave CS_KEY_VAL nel registro CS->KEY.

Anche per i clock è possibile usare dei divisori che fungono da prescaler.

9) Embedded C

Software independence

La software independence rappresenta il tentativo di sviluppare il maggior numero possibile di software con indipendenza dall'architettura e indipendenza dalla piattaforma.

L'obiettivo primario di questo approccio è massimizzare la portabilità e la riusabilità del software.

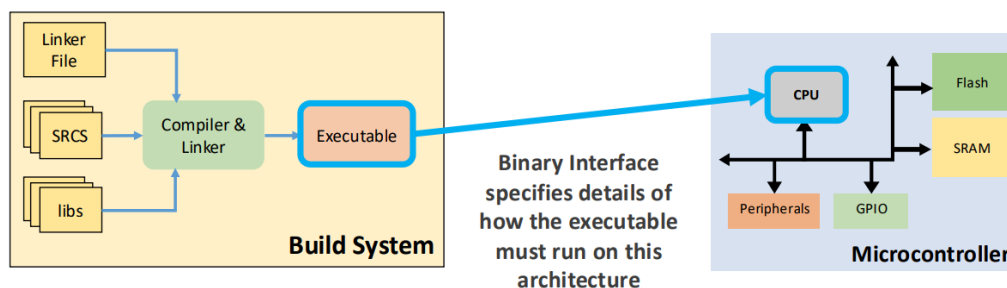
Tuttavia bisogna notare che è impossibile rendere ogni componente del software completamente indipendente.

Il livello di software superiore (come gli strati application, device lib) può essere riutilizzato tra piattaforme diverse. Per esempio gli strati di uart lib non può essere riutilizzato. Però gli strati firmware devono interagire con l'hardware, e il codice assembly ad esempio è intrinsecamente dipendente dall'architettura.

EABI = Embedded Application Binary Interface, è una specifica che fornisce dettagli essenziali su come un file binario deve essere compilato e interfacciato con i componenti di una piattaforma embedded. In pratica le EABI specificano dettagli critici che consentono al sistema di compilazione (compilatore e linker) di produrre un eseguibile funzionante sull'architettura target.

Le EABI specificano:

- l'uso dei registri, e la dimensione della Parola (Word Size)
- Requisiti di Archiviazione di Codice/Dati, ossia come il codice e i dati devono essere disposti nella memoria.
- Modalità di indirizzamento
- Convenzioni di chiamata (come le funzioni vengono chiamate, come i parametri vengono passati...)
- Funzioni di supporto e librerie.



Architettura

Ricorda che **istruzione**= unità fondamentale di lavoro o operazione (può essere aritmetica, logica, di controllo del flusso, di tipo load/store).

Word= Taglia fondamentale degli operandi in ogni operazione.

La size delle istruzioni può variare, nell'ARM instruction set ARMv6-M : 16 bit e 32 bit.

I tipi di variabili di lunghezza variabile (Variable Length types) possono causare problemi di portabilità tra architetture e compilatori diversi.

Per risolvere questo, si utilizzano tipi di dati esplicitamente definiti che specificano l'esatta dimensione di archiviazione (in bit) e il segno, garantendo così la portabilità.

Questi tipi standardizzati sono definiti nel file header stdint.h.

```
#ifndef __STDINT_H__
#define __STDINT_H__

/* 8-bit signed/unsigned Integers */
typedef signed char int8_t;
typedef unsigned char uint8_t;

/* 16-bit signed/unsigned Integers */
typedef signed short int int16_t;
typedef unsigned short int uint16_t;

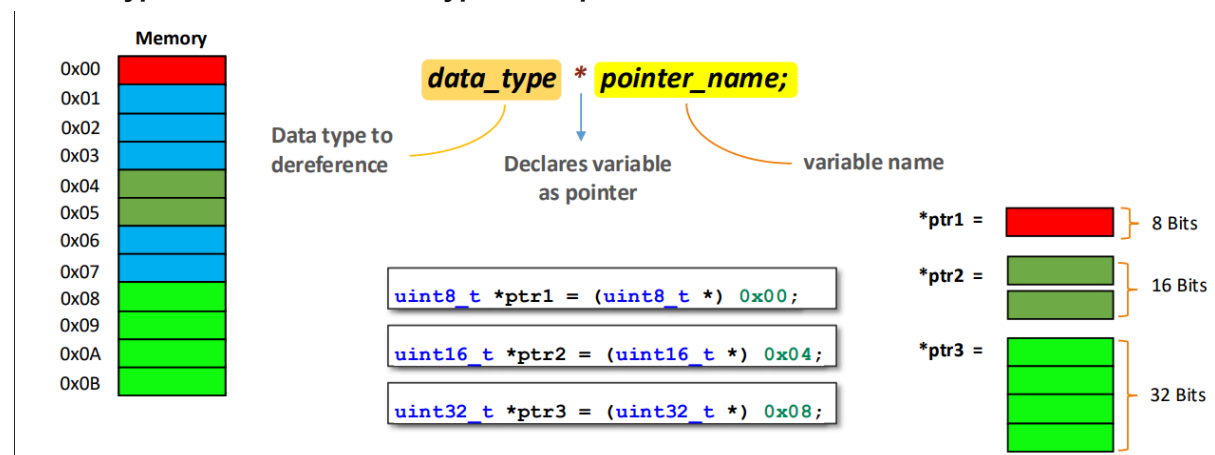
/* 32-bit signed/unsigned Integers */
typedef signed long int int32_t;
typedef unsigned long int uint32_t;

/* 64-bit signed/unsigned Integers */
typedef signed long long int int64_t;
typedef unsigned long long int uint64_t;

#endif /* __STDINT_H__ */
```

La keyword typedef permette al programmatore di creare i propri tipi, facendo alias. Si può applicare sia a tipi standard che a tipi derivati.

Pointer types = denotano il datatype che il puntatore dereferenzierà



Pointer size

La size dei puntatori è sempre la stessa! Ossia 32-bit. Non cambia nulla se sono puntatori a interi da 8 bit o da 16 bit. Siccome i puntatori contengono l'indirizzo di 32-bit di una locazione in memoria, ciò significa che **lo spazio indirizzabile è $2^{\text{pointer_Length}}$** . (in ARM: $2^{32} \text{ bit} = 4 \text{ GB}$).

Di base gli indirizzi di memoria non sono interi, quindi per creare un puntatore a intero ad esempio devo fare un cast esplicito per trasformare un intero in un indirizzo di memoria utilizzabile dal processore.

```
uint16_t * ptr = (uint16_t *) 0x480C0000;
```

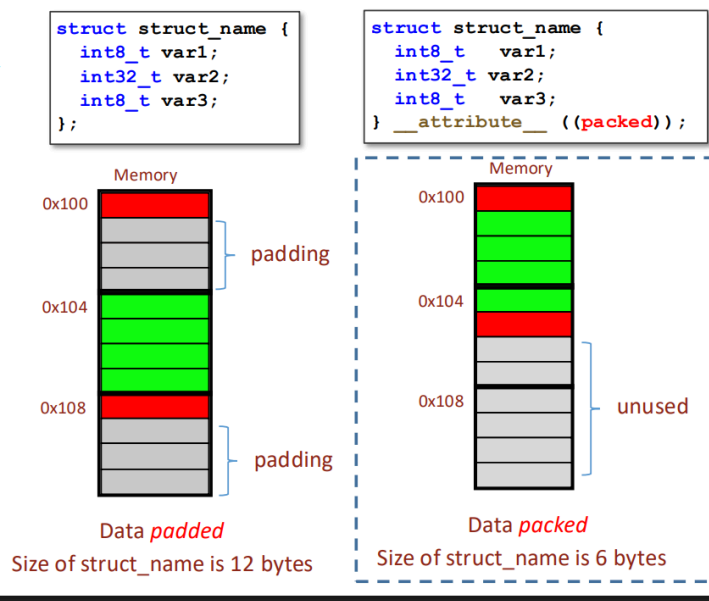
Casts Integer to address

Questa cosa che stiamo facendo è creare un puntatore a un tipo di dato in una locazione di memoria fisica, ad esempio per modificare/manipolare i registri.

Come sono piazzate/allineate le variabili in memoria?

1. **CPU-efficient (memory inefficient - data aligned)** : Le variabili sono piazzate in locazioni che sono multiple della dimensione del dato stesso. Per esempio un byte può essere posizionato in qualunque indirizzo, una half word può essere allineata a indirizzi multipli di 2(0x100,0x102,0x104...), una word a indirizzi multipli di 4(0x100,0x104,0x108...). Ciò rende la CPU efficiente, perché le operazioni di Load/Store avvengono in modo efficiente, infatti la CPU non legge la memoria un byte alla volta, ma accede a blocchi di 4 bytes. La CPU può caricare o memorizzare i dati di una specifica dimensione con una singola istruzione, perché questi dati risiedono interamente in un blocco di memoria accessibile in modo diretto. Questo metodo è però inefficiente per la memoria, perché spesso introduce padding.
2. **Memory-efficient (CPU inefficient - data packed)**: Non c'è padding, i dati non sono allineati. Qui l'operazione di Load/Store diventa molto più complessa e costosa in termini di cicli. Se il dato è non allineato il processore deve eseguire più operazioni:
 - prima carico la parte inferiore in un registro
 - poi carico la parte superiore in un altro
 - infine combino i due registri

Occupazione delle struct in memoria:



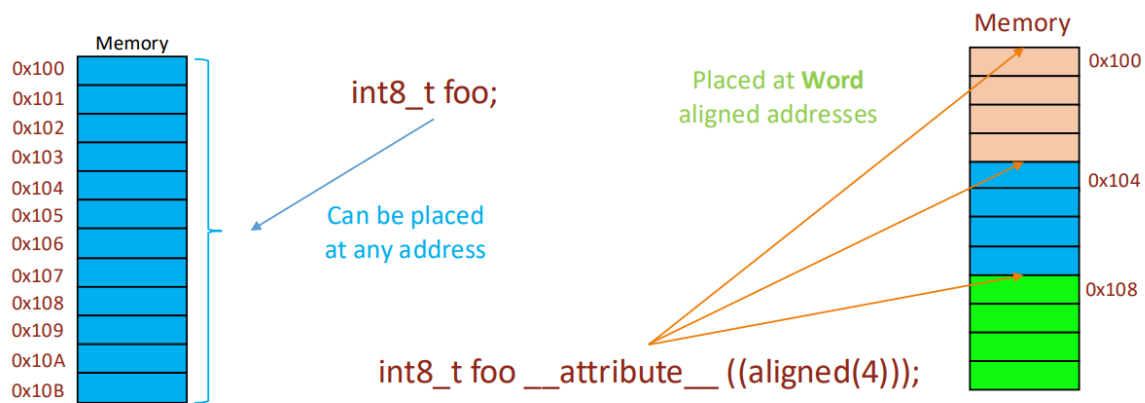
Qui vediamo che nel caso di data alignment, la struct occupa in tutto 12 bytes: Il primo campo è un byte e può essere messo in 0x100, il secondo campo è una parola di 4 bytes e può essere messo solo in 0x104, il terzo campo è un byte e può essere messo in 0x108.

Invece nel caso di data packed, la stessa struct usa solo 6 bytes. Si può impostare questo modo con l'attributo del compilatore "packed".

Attributi del compilatore: Gli attributi permettono di dare dettagli specifici su come compilare il codice per variabili, structs, funzioni ecc.

Nota però che gli attributi non sono parte del C-standard, quindi non portabili tra compilatori diversi.

Attributi di allineamento: Specificano l'allineamento dei dati in memoria. Con `aligned(x)`, dove `x` è una potenza di 2, io posso scegliere in quali indirizzi multipli di due andranno memorizzate le variabili.



Esempio in cui allineo a multipli di 4 tutti i campi della struct:

<pre>typedef struct { int8_t var1; int32_t var2; int8_t var3; } str1;</pre>		<pre>typedef struct { int8_t var1 __attribute__((aligned(4))); int32_t var2 __attribute__((aligned(4))); int8_t var3 __attribute__((aligned(4))); } str2;</pre>
---	--	---

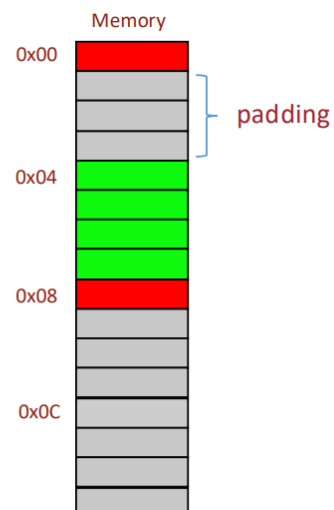
sizeof(str2) = 12 Bytes

Esempio in cui allineo la struct.

- Aligned structure members

```
typedef struct {
    int8_t    var1;
    int32_t   var2;
    int8_t    var3;
} str3 __attribute__((aligned));
```

sizeof(str3) = 9 Bytes



Attributi di funzione:

Gli attributi del compilatore si possono applicare alle funzioni. In C99 ad esempio esiste la parola chiave **inline**, ma questa è solo un suggerimento per il compilatore, e potrebbe essere ignorato. L'attributo **always_inline** invece forza il compilatore a eseguire l'inlining, ovvero a copiare il corpo della funzione direttamente nel punto in cui viene chiamata, saltando la convenzione di chiamata standard. In questo modo si elimina l'overhead della chiamata a funzione (salvataggio registri, salto e ritorno) e si velocizza l'esecuzione, ma aumenta la dimensione del codice binario (codice duplicato ogni volta che funzione viene chiamata). Buono per funzioni piccole.

Function Pragmas:

Le direttive `#pragma` sono istruzioni speciali per controllare il processo di compilazione dal codice sorgente stesso, invece che dalla riga di comando. Ad esempio posso usare i pragmas per isolare una funzione e compilare solo quella con ottimizzazione zero (O0), mentre il resto del programma rimane ottimizzato.

Only this code will have Zero Optimizations applied

```
#pragma GCC push
#pragma GCC optimize ("O0")
int32_t add( int32_t x, int32_t y )
{
    return (x + y);
}
#pragma GCC pop
```

Altro esempio d'uso :dire di dare errore durante la compilazione se il codice usa qualcuna tra le funzioni specificate: `#pragma GCC poison printf sprintf`.

Oppure si può dire di compilare una funzione con un'architettura specifica:

`#pragma GCC target ("arch=armv6") -or- ("cpu=cortexm0plus")`

Direttive del preprocessore:

`#ifndef X` → entra se X NON è definita

`#ifdef X` → entra se X è definita

`#if condizione` → entra se condizione è vera

`#elif condizione` → else if

`#else` → else

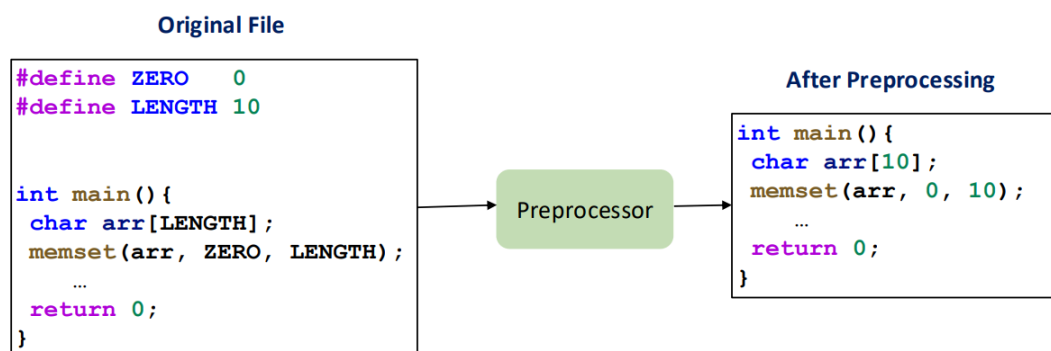
`#endif` → chiude

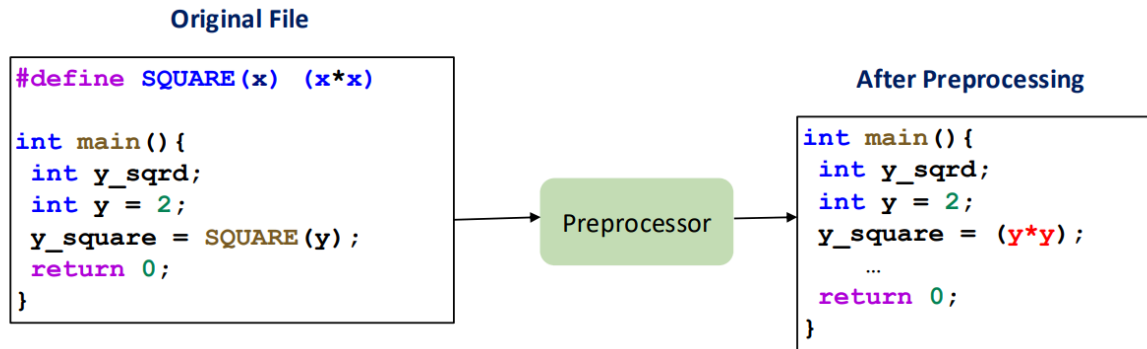
Output del preprocessore:

Lo si può vedere nel file `.i` compilando il sorgente così :

\$ gcc -E -o main.i main.c

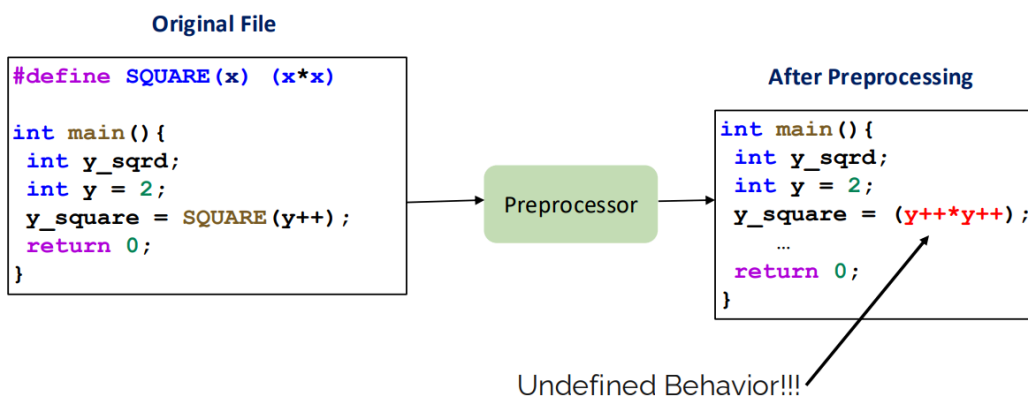
Di base nel file `.i` rispetto al sorgente vedo che al posto delle variabili lui sostituisce i valori specificati nelle direttive:





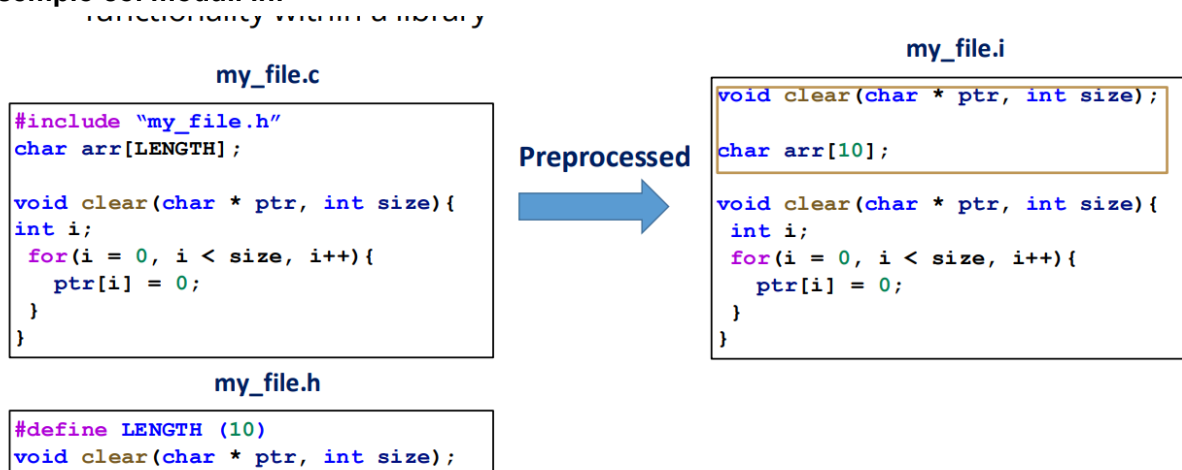
Esempio con comportamento indefinito:

Macro Substitution



Qui è un casino perchè se guardo il sorgente mi aspetto che square(2++) mi restituisca 4. Ma in realtà mi restituirà 6, perchè fa 2*3 (y++ viene aumentato dopo il * e da due diventa 3), e alla fine del codice y sarà 4.

Esempio coi moduli .h:



Compile-Time Switch: E' una tecnica che permette di dettare al compilatore quale porzione di codice deve essere compilata e inclusa nell'eseguibile finale. E' fondamentale per scrivere codice portabile che funzioni su hardware diversi e per il debugging. Lo si fa con le solite direttive #ifndef #if #endif ecc..

Esempio Pratico: Se si sta sviluppando codice che deve girare sia sulla piattaforma **KL25z** che sulla **MSP432**, si può usare uno switch per includere l'header file corretto per la UART specifica dell'hardware:

In questo modo, il compilatore "vede" solo il codice rilevante per la piattaforma scelta, ignorando l'altro...

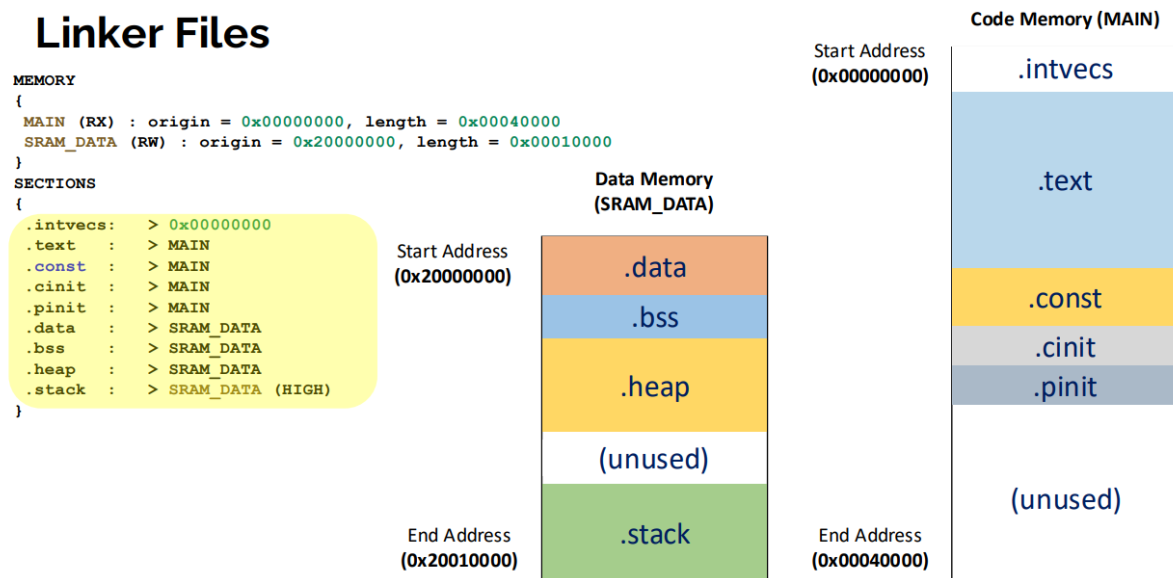
Includere librerie precompilate:

Prima di farlo devo chiedermi se è compilata e se è ottimizzata per la mia architettura.

Librerie statiche : Sono direttamente linkate nell'eseguibile di output, quindi sono installate con l'immagine del programma come parte dell'eseguibile. Il collegamento avviene a tempo di compilazione.

Librerie dinamiche: Sono collegate solo a runtime all'eseguibile, quindi l'eseguibile non contiene il codice della libreria, ma solo i riferimenti per trovarla. Queste librerie sono spesso preinstallate sul target, risiedono in memoria separatamente rispetto all'immagine. Sono tipicamente usate in applicazioni che girano su un sistema operativo, che gestisce il caricamento dinamico delle risorse.

Il **linker** prende in input multipli eseguibili e li unisce in uno unico. L'entry e l'exit point del main sono inclusi in una libreria standard che viene automaticamente inclusa dal linker, ma posso stoppare l'autolink delle standard libs con -nostdlib flag. Dopo il linking , i simboli sono risolti. Nei linker file si trovano anche i dettagli su come vengono mappati i dati nelle varie regioni di memoria fisica:



10)MSP432 Memory mapped I/O details

Nel msp tutte le risorse(flash, RAM, periferiche) condividono lo stesso spazio di indirizzamento di 4GB. La Memory Map è una convenzione che assegna intervalli di indirizzi specifici a diverse componenti fisiche.

Le periferiche (Timer, GPIO, UART ecc.) si trovano in una regione dedicata , a partire dall'indirizzo 0x4000000.

Per comunicare con una periferica, la CPU legge o scrive dati in questi indirizzi di memoria come se fossero normali variabili, ma invece di salvare dati in RAM, queste operazioni modificano la configurazione hardware o leggono lo stato dei sensori.

I **Core CPU registers** sono i registri situati direttamente all'interno del core: **questi non fanno parte delle mappa di memoria standard.**

Mentre per accedere alla RAM o alle periferiche si usano indirizzi di memoria, per accedere ai COre register si utilizzano istruzioni assembly speciali (es : MOV R0, #1).

Il banco dei registri nel core è composto da 16 registri (r0-r15):

- R0-R12 = General purpose (usati ad esempio per timer)
- R13 = Stack pointer SP
- R14 = Link Register LR
- R15 = Program Counter PC

Il registro PSR (Program Status Register)

E' un registro speciale che cattura lo stato corrente del processore e dell'applicazione. In realtà è una vista combinata di 3 sotto-registri interni:

1. **APSR** = Contiene i flag di stato risultanti dalle operazioni aritmetiche o logiche precedenti, fondamentale ad esempio per le istruzioni condizionali.
2. **IPSR** = Indica se attualmente il processore sta eseguendo un'interruzione e quale (contiene il numero dell'eccezione corrente).
3. **EPSR** = Gestisce lo stato di esecuzione dell'architettura, contiene il bit T (Thumb state).

Memory Access Macros

Siccome le periferiche sono memory-mapped, in C si usano le memory access macros per scrivere / leggere dati a indirizzi di memoria fisici specifici.

Senza le macro, in C per scrivere il valore 0x0202 in un registro all'indirizzo 0x4000000 (quello del timer A0), dovrei scrivere un codice complesso come questo :

```
*((volatile uint16_t *)0x40000000) = 0x0202;
```

Invece io per astrarre la complessità posso definire delle Macro che nascondono la sintassi dei puntatori:

```
#define HWREG16(x) (*((volatile uint16_t *) (x)))
```

x: è l'indirizzo fisico, tipo 0x40000000, che rappresenta l'indirizzo di memoria del registro hardware.

(uint16_t *): Inizialmente il compilatore vede x solo come un numero intero, questo cast dice al compilatore di trattare il numero come un puntatore (indirizzo) ad un dato a 16 bit.

volatile : Dice al compilatore : "il valore di questo indirizzo può cambiare in qualsiasi momento, quindi non ottimizzare questo accesso.

*****: La dereferenziazione: accede al contenuto puntato, così posso assegnargli un nuovo valore. Poi faccio ad esempio:

```
#define TA0CTL (HWREG16(0x40000000));
```

```
TA0CTL = 0x0202; //qui ho scritto nel registro come se fosse una variabile normale
```

Example Use of Access Macro:

```
TA0CTL = 0x0202;      ↔      (* (volatile uint16_t *) 0x40000000) = 0x0202;
```

Bitwise operators:

```
uint8_t * ptr = (uint8_t *) 0x1000;  
  
Set 4th bit: *ptr |= 0x10;  
Clear 4th bit: *ptr &= ~(0x10);  
Toggle 4th bit: *ptr ^= 0x10;
```

0x10 = 0001 0000 = 16

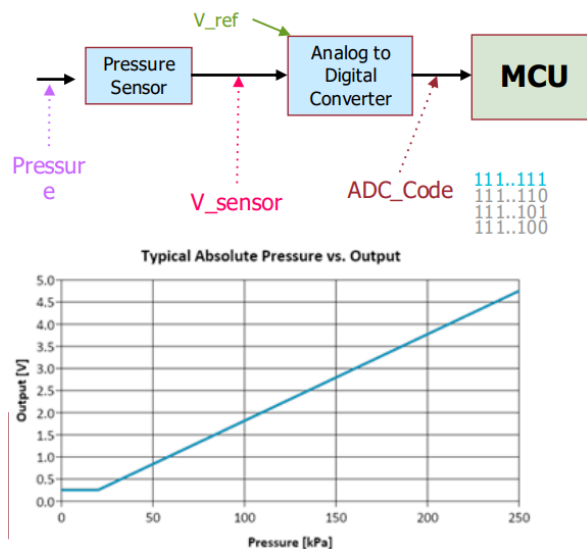
12) Analog to digital conversion

Spesso i sistemi embedded devono misurare valori di parametri fisici. Questi parametri sono spesso continui (analogici) e non in una forma digitale (discreta) che i computer riescono a processare. Entrano così in gioco i sensori, come quello di temperatura, luminosità, pressione, accelerazione.

Esempio: Sensore analogico che misura la profondità sott'acqua

Calcoliamo quanto un oggetto è sott'acqua così:

- Un sensore di pressione rileva un certo valore di pressione e genera un valore di voltaggio proporzionale (V_{sensor}).
- Un convertitore da analogico a digitale ADC prende in input il valore V_{sensor} e genera un codice digitale intero proporzionale.



Concetto fondamentale: La conversione analogico digitale non misura un voltaggio in termini assoluti, ma relativi. L'ADC risponde alla domanda "Quanto è grande il voltaggio in ingresso V_{in} rispetto al voltaggio di riferimento V_{ref} "?

Quindi l'ADC code rappresenta questa frazione di V_{in} su V_{ref} .

Sampling = Campionare il segnale continuo ogni intervallo di tempo ΔT , rendendo così il segnale discreto nel tempo.

Quantizzazione = Ogni campione ottenuto dal sampling viene approssimato al livello più vicino all'interno di una scala di valori finiti predefinita, così poi ho un insieme di valori finiti e posso codificarli in codice binario.

Frequenza di sampling = La frequenza di campionamento deve essere almeno uguale al doppio della massima frequenza presente nel segnale.

Un sistema di conversione analogico->digitale tipicamente consiste in 3 stadi:

1. **Multiplexer** : riduce il numero di componenti ADC richieste
2. **Sample and hold** : dispositivo che ha la funzione di trattenere per un pò di tempo il valore corrente del segnale analogico di input fino a che una conversione digitale numerica di quel valore è completata.
3. **ADC** = converte da analogico a digitale

Alcune metriche per misurare la performance dell'ADC sono:

1. **Il numero di bit** usati per generare i codici binari : + è alto più è accurato
2. **Il tempo di conversione**: dall'inizio della conversione alla generazione del risultato.

MSP432 ADC Overview:

L'ADC nel msp432 ha un **accuracy di 14 bit**, ossia può convertire un segnale analogico in un valore digitale compreso tra 0 e $2^{14}-1$ (16383) -> precisione molto alta rispetto ad altri microcontrollori.

Riesce a campionare i segnali con una frequenza di **1 milione di campioni al secondo**.

Il sistema gestisce fino a **32 canali di ingresso** tramite un multiplexer analogico. La maggior parte dei canali (da A0 a salire) sono mappati su pin fisici del microcontrollore per misurare sensori esterni, invece i canali più alti sono collegati a sensori interni al chip : come il **sensore di temperatura** per monitorare la temperatura del chip e il **monitor di batteria** per misurare la tensione di alimentazione.

Modalità differenziale : E' possibile configurare coppie di ingressi per leggere la differenza di tensione tra due pin, invece che la tensione rispetto a terra.

Gestione della memoria: Una caratteristica distintiva della msp432 è la sua gestione avanzata della memoria per i risultati. Invece di un solo registro "risultato" , il modulo dispone di **32 registri di memoria dedicati (ADC14MEM0 - ADC14MEM32)**.

Ogni locazione di memoria può essere configurata individualmente per leggere uno specifico canale d'ingresso (es. MEM0 legge A0, MEM1 legge il sensore di temperatura ecc..).

Voltaggio di riferimento:

L'MSP può usare come voltaggio di riferimento Vref:

- un voltaggio fornito su un pin esterno per la massima precisione
- un voltaggio interno programmabile che può fornire 1.2V, 1.45V, o 2.5V)
- oppure può usare come riferimento la tensione di alimentazione Vcc

Window Comparator:

Per l'efficienza energetica, l'ADC 14 include un comparatore hardware a finestra: è impossibile impostare 2 soglie : High Threshold e Low Threshold.

L'ADC monitora il segnale e solo se il valore misurato entra in questa finestra genera un interrupt.

Il vantaggio è che la CPU può rimanere in modalità sleep mentre l'ADC monitora il sensore, svegliandosi solo quando il valore diventa "interessante".

Clock e trigger:

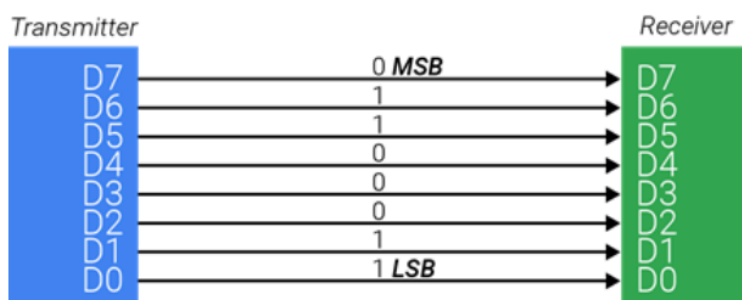
L'ADC può essere pilotato da diverse sorgenti di clock e dispone di divisori interni per adattare la velocità di campionamento.

13) Embedded I/O

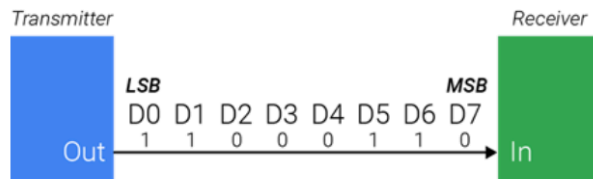
In un sistema embedded tipicamente si hanno molti tochi / sensori / moduli, bisogna integrarli tra loro e farli comunicare.

Metodi di comunicazione:

Parallela



Seriale:



This approach **reduces number of wires**

Seriale vs parallelo

Spesso non è fattibile l'invio di tutti i bit allo stesso momento a causa di:

- Costi e peso: ci vogliono + fili, e più grandi
- Affidabilità: con tanti fili è più facile avere fail
- Complessità temporale : alcuni bits possono arrivare dopo rispetto ad altri a causa delle variazioni nella capacità / resistenza attraverso i conduttori.

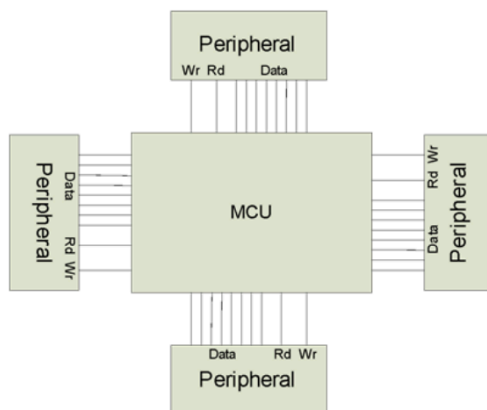
La comunicazione seriale costa meno rispetto alla parallela, ed è anche più affidabile, e riduce i problemi di "clock skew" = sfalsamento del clock.

Inoltre , siccome è più affidabile, la comunicazione seriale può avere un clock a più alta frequenza , quindi produce anche più throughput.

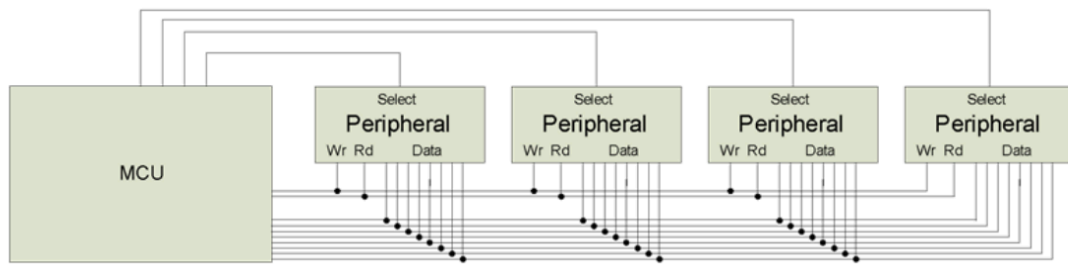
Tuttavia la conversione tra seriale e parallelo può provocare overhead aggiuntivo.

Esempi di connessioni:

Connessioni punto a punto: tante connessioni parallele tra il MCU e ogni periferica (con linee per dati, scrittura e lettura) : è molto veloce e permette trasferimento simultaneo, ma richiede molte connessioni e una PCB grande (si scala male).

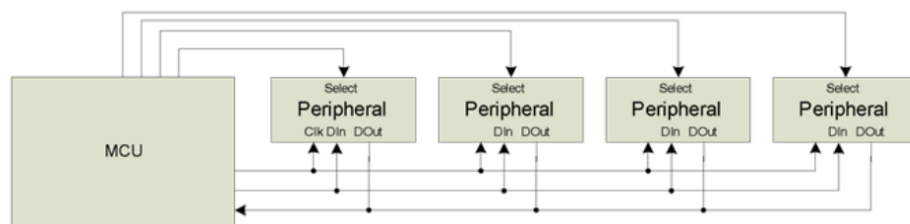


Bus paralleli: Tutti i dispositivi usano i bus per condividere i dati, leggere e scrivere.
La MCU richiede meno pins per i dati, ma può comunicare solo con una periferica alla volta.



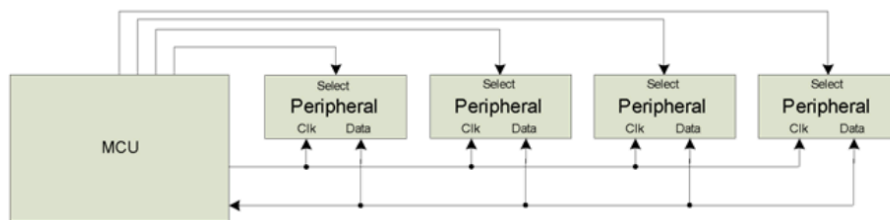
Synchronous Full-Duplex Serial Data Bus:

Permette simultanei send e receive.



Synchronous Half-Duplex Serial Data Bus

Non permette simultanei send e receive.



3 Protocolli di comunicazione seriale:

UART	I2C	SPI
<i>universal asynchronous receive-transmit</i>	<i>inter-integrated circuit</i>	<i>serial peripheral interface</i>
no shared clock for synchronicity	shared clock	shared clock
slower	intermediate speed	high transfer speed
simple wiring	bi-directional but one-at-a-time	simultaneous full duplex
	simple wiring	complex wiring

UART (Universal Asynchronous Receive-Transmit)

Nella comunicazione seriale di tipo asincrono, si elimina la linea del clock.

Il trasmittente deve aggiungere uno **start bit** (sempre lo stesso valore) per indicare l'inizio di ogni data frame.

Il ricevitore detecta il fronte di discesa dello start bit e lo usa come riferimento temporale per campionare la linea dati a intervalli regolari. E' importante che tutte le periferiche usino gli stessi parametri di comunicazione (come la velocità di comunicazione ad esempio 9600 baud).

UART è l'hardware periferico che gestisce questo tipo di comunicazione.

Funzione: converte i dati paralleli della CPU in seriali per la trasmissione nel cavo seriale e riassembla i dati seriali ricevuti in parallelo.

Cablaggio: Utilizza fili separati per la trasmissione(TX) e la ricezione (RX).

Il protocollo:

I dispositivi comunicanti devono concordare tra loro un protocollo /schema preciso:

1. Decidere l'Idle State: quando non c'è trasmissione, la linea è mantenuta a livello logico alto (1)
2. Start bit: Bisogna decidere se è 0 o 1 il primo bit mandato (ad esempio la trasmissione inizia portando la linea a livello basso per un ciclo di clock)
3. Data bits: Dopo lo start bit si mandano i bit di dati (di solito 8 bit) in sequenza dal meno significativo al più significativo
4. Parity Bit(opzionale): un bit aggiuntivo aggiunto in coda ai data bits per controllo errori. Un bit di parità può dire solo se un numero dispari di bit è stato corrotto, ma non rileva un numero pari di bit corrotti.
5. Stop Bit: Alla fine la linea viene riportata a livello alto (1) per indicare la fine del trasferimento

E' importante che i clock del trasmittente e del ricevitore siano abbastanza vicini, dal momento in cui non c'è clock condiviso e l'unico riferimento è lo start bit.

Sul controllore MSP432 l'UART è gestita dal modulo **eUSCI**.

I2C (Inter-Integrated Circuit)

L'I2C è un protocollo di comunicazione seriale, sincrono e a commutazione di pacchetto, progettato per velocità intermedie e brevi distanze.

Il bus I2C ha due linee di segnale: l'**SCL(Serial Clock line)** e l'**SDA (Serial data line)**.

Open drain: I dispositivi collegati al bus I2C usano transistor "open-drain". Questo significa che un dispositivo può tirare giù la linea a livello basso (0) , ma non può spingerla a livello alto. Per avere un livello logico alto (1) , sono necessarie delle resistenze di pull up esterne collegate all'alimentazione che mantengono le linee alte quando nessun dispositivo sta trasmettendo.

Ruoli:

Master : E' il dispositivo che inizia la comunicazione e genera il segnale di clock (SCL). Solitamente è il microcontrollore MCU.

Slave: E' il dispositivo che viene indirizzato dal master. Gli slave non generano il clock; si limitano a rispondere alle richieste.

Sullo stesso bus possono esserci molti slave, ognuno identificato da un indirizzo univoco. Il protocollo supporta anche più master, ma devono gestire l'arbitraggio per non parlare contemporaneamente.

Il protocollo di comunicazione:

1. **Condizione di START:** Il Master segnala l'inizio della trasmissione portando la linea SDA da Alto a Basso mentre la linea SCL rimane Alta. Questo è l'unico momento in cui SDA cambia mentre SCL è alto (infatti solitamente i dati cambiano quando SCL è basso).
2. **Indirizzamento:** Il master invia **7 bit** che rappresentano l'indirizzo dello Slave con cui vuole parlare. Aggiunge poi un 8° bit di R/W che dice se vuole scrivere o leggere dallo slave.
3. **ACK/NACK:** Al 9° impulso di clock, il trasmettitore rilascia la linea SDA. Se lo slave ha riconosciuto il proprio indirizzo, **tira giù la linea SDA** per inviare un ACK. Se la linea rimane alta, significa NACK.
4. **Trasmissione Dati :** I dati vengono inviati in pacchetti di 8 bit. Dopo ogni byte, il ricevente deve inviare un bit di ACK per confermare la ricezione.
5. **Condizione di stop:** Il Master termina la comunicazione portando la linea SDA da Basso a Alto mentre SCL è Alto.

Il cablaggio è semplice (solo 2 fili), ma è più lento dell'SPI. Inoltre c'è overhead dovuto al fatto che bisogna inviare l'indirizzo dello slave e i bit di ACK. La comunicazione è bidirezionale, ma può avvenire in una sola direzione alla volta (non simultaneamente come nell'SPI).

SPI(Serial Peripheral Interface)

L'SPI è sincrono e Full-Duplex (può trasmettere e ricevere contemporaneamente). E' progettato per trasferimento di dati ad alta velocità su brevi distanze tra Master e Slaves.

L'SPI usa **4 fili** per collegare i dispositivi:

1. **SCLK** = La linea di Clock, generata sempre da Master
2. **MOSI** (Master Out Slave In): la linea dati per inviare informazioni **dal Master allo Slave**
3. **MISO** (Master In Slave Out) : da Slave a Master
4. **CS** (Chip Select) o **SS**(Slave Select) : Una linea dedicata per abilitare uno specifico Slave.

Il Master controlla il bus e inizia tutte le comunicazioni. Gli Slave non possono iniziare a parlare spontaneamente; rispondono solo quando interrogati.

Selezione dello Slave: A differenza dell'I2C che usa indirizzi software inviati sul bus, l'SPI usa un indirizzamento hardware: il master ha una linea CS dedicata per ogni Slave connesso. Per parlare con un dispositivo, il Master porta la linea CS di quello specifico Slave a livello basso (Active Low). Tutti gli altri slave con CS alto ignorano il bus.

E' molto più veloce di I2C e UART, ed è pure semplice, non ci sono bit di Start, Stop o ACK da gestire. Lo svantaggio è il cablaggio complesso : per cablare N dispositivi servono 3 + N linee), e non c'è nessun feedback con gli ACK.

Protocol	Speed	Signals Req. for Bidirectional Communication with N devices	Device Addressing
UART (Point to Point)	Fast – Tens of Mbit/s	2*N (Tx/D, Rx/D)	None
SPI	Fast – Tens of Mbit/s	3+N for SCLK, MOSI, MISO, and one CS per device	Hardware chip select signal per device
I ² C	Moderate – 100kbit/s, 400 kbit/s, 1Mbit/s, 3.4Mbit/s. Packet overhead.	2 (SCL, SDA)	In packet

16) Accelerometro

Nel kit MSP432 BoosterPack, l'accelerometro è trattato come un sensore analogico. Gli accelerometri sono dispositivi elettromeccanici che misurano l'accelerazione di un oggetto (quanto cambia la sua velocità).

L'interfaccia utilizzata per connettere l'accelerometro al microcontrollore è il Pin Analogico.

Funzionamento: Il sensore rileva l'accelerazione e genera una tensione di uscita proporzionale al valore fisico misurato. Il microcontrollore deve quindi convertire questa tensione in un numero digitale.

Il sensore rileva l'accelerazione lungo i 3 assi X, Y, Z e genera 3 tensioni di uscita proporzionali ai valori fisici misurati.

Lab : Simple Activity Recognition

Vogliamo capire ad esempio se una persona cammina o è ferma o sale scale ecc...

1. **acquisizione dati** : l'accelerometro fornisce dati grezzi sui tre assi. Un singolo dato non è sufficiente a capire un'attività, quindi il sistema raccoglie un gruppo di campioni nel tempo, con la funzione `acquire_window()`. Questa riempie un buffer con un numero prefissato di letture. Poi viene chiamata, per il campionamento, la funzione `accel_sample(&sample)` che legge i registri fisici `readEccelX`, `readAccelY`, `readAccelZ` per ottenere i valori istantanei degli assi.
2. **Trasformazione/filtraggio** : I dati grezzi dell'accelerometro sono spesso rumorosi. Anche se il dispositivo è fermo si potrebbero leggere valori piccoli al posto di 0. Quindi si definisce una soglia di rumore, si itera su tutti i campioni, se il valore è inferiore alla soglia lo si forza a zero.
3. **Estrazione delle feature**: Si calcola la media, la deviazione standard e il magnitudo
4. **Train del modello**: Si usano queste funzioni per trainare un modello, così impara a riconoscere quel movimento registrato.

17) The Internet of Things (IoT)

Le Things possono essere qualsiasi cosa nel mondo. E' un vasto ecosistema di dispositivi e oggetti intelligenti in azione sopra ad un'infrastruttura di rete.

L'IoT rappresenta l'evoluzione dei sistemi embedded in sistemi embedded in rete.

Differenze rispetto a sistemi embedded classici

Sono connessi a internet, hanno maggiori funzionalità e + intelligenti, scambiano informazioni su un'infrastruttura di rete globale.

L'evoluzione è stata resa possibile grazie a diversi progressi tecnologici:

- Hardware più economico, veloce e a basso consumo
- Dispositivi più piccoli e sensori integrabili ovunque, inclusi quelli indossabili
- L'adozione di standard di comunicazione diversificati (es: Bluetooth o Zigbee).
- Dati e Cloud: L'uso dei servizi Cloud per processare l'enorme mole di informazioni generate dai sensori, che i piccoli dispositivi non potrebbero elaborare da soli.

Nuove architetture:

Mentre un sistema embedded classico elabora tutto localmente, l'IoT distribuisce l'intelligenza:

- **Cloud computing**: Si elabora su server potenti, e i dispositivi embedded sono semplici raccoglitori di informazioni
- **Fog Computing**: Si spinge l'intelligenza più vicino al dispositivo, per ridurre latenza e traffico di dati
- **Edge computing**: Riporta l'elaborazione direttamente sul dispositivo embedded per risposte immediate e minore dipendenza dalla rete (es: automotive).

Connectivity:

Le tecnologie utilizzate sono varie e vengono scelte in base a compromessi come raggio d'azione, consumo energetico, velocità di trasmissione e costo.

Corto raggio : Bluetooth, Zigbee

Medio raggio: WIFI

Lungo raggio: LPWAN , Cellular

Bluetooth Low Energy (BLE):

Il BLE è una revisione del protocollo Bluetooth progettata per IoT e per dispositivi con vincoli energetici stringenti.

E' progettato per funzionare con batterie a bottone (coin cells) per lunghi periodi, ha una procedura di connessione più rapida del bluetooth classico e un raggio di trasmissione ridotto. Si basa sul concetto di spostare il carico energetico dal dispositivo IoT al dispositivo di calcolo (come lo smartphone). Supporta comunicazioni Many-to Many.

Zigbee:

E' una tecnologia Low-power-Low-Rate, in un raggio tra 10-100 m. Zigbee è adatto per applicazioni in cui i messaggi sono brevi e meno frequenti (ad es. la domotica).

MQTT (MQ Telemetry Transport):

E' un protocollo di comunicazione leggero basato sul modello **publish/subscribe** progettato per l'IoT per distribuire messaggi brevi tra dispositivi. E' adatto per reti con bassa larghezza di banda e alta latenza. L'architettura non è punto-punto, ma si centra sul **Broker**(server centrale che riceve tutti i messaggi e li smista).

Il publisher pubblica un messaggio su un canale (Topic). I subscriber si iscrivono a quel topic, il broker manda il messaggio a tutti gli iscritti a quel topic.

Vantaggio : i client non conoscono gli indirizzi IP gli uni degli altri, ma solo quello del broker.

18)Embedded software testing

Rule of Thumb: Testa il più che puoi nell'host, non testare sul target più di quanto dovresti. Testare nell'host permette di usare strumenti di debug potenti, trovare i bug logici rapidamente e isolare i problemi software da quelli hardware.

Black box testing: Si testa il sistema senza conoscerne il codice interno, basandosi solo su input e output attesi. Utile per trovare errori che sfuggono allo sviluppatore.

Clear box testing: Si progettano test conoscendo la struttura interna del codice. L'obiettivo è coprire tutti i percorsi possibili nel codice.

Per applicare la Thumb Rule si può fare così :

1. Si **separa** il codice in 2 parti: una dipendente dall'hardware(driver / registri) e una indipendente (logica / algoritmi).
2. Crea un' **interfaccia** pulita tra le due
3. **Scaffolding(impalcatura):** Quando testi su PC, sostituisci le funzioni che accedono all'hardware reale con funzioni finte che simulano il comportamento dell'hardware con valori noti.

Testing degli interrupt: Difficili da testare a causa della concorrenza. La best practice qui è dividere l'ISR in due parti : una minima che interagisce con l'hardware e una che processa il dato. Quest'ultima parte di processamento non deve avere dipendenze hardware, così può essere chiamata e testata intensivamente sul PC simulando l'arrivo di dati senza generare interrupt fisici reali.

Assert Macro: E' buona pratica usare la macro assert(condizione) nel codice. Se l'assunzione è vera, non succede nulla, se è falsa il programma si stoppa. Su un sistema embedded questo può ad esempio far lampeggiare un led specifico o entrare in un loop infinito sicuro.

Strumenti Hardware e Software:

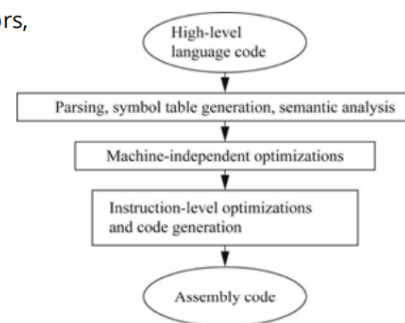
Poiché non è possibile testare tutto su PC si usano strumenti specifici:

- **Logic Analyzer:** Mostra graficamente i segnali digitali 0/1 nel tempo.
- **Oscilloscopio:** Per verificare l'integrità fisica del segnale e problemi di rumore o livelli di tensione errati.
- **Voltmetro:** Per controlli base tipo se i pin sono collegati
- **Profilers:** Per analizzare quale parte del codice è stata eseguita e quante volte -> utile per ottimizzazione
- **Instruction Set Simulators:** Simulatori che girano su PC ma mimano il comportamento istruzione per istruzione del processore target.

19) Programming for efficient computing

Review of Compiler Stages

- Parser
 - reads in source code e.g. C code, checks for lexical and syntax errors,
 - forms intermediate code (tree representation)
- High-Level Optimizer
 - Modifies intermediate code (processor-independent)
- Code Generator
 - Creates assembly code step-by-step from each node of the intermediate code, Allocates variable uses to registers
- Low-Level Optimizer
 - Modifies assembly code (parts are processor-specific)
- Assembler
 - Creates object code (machine code)
- Linker/Loader
 - Creates executable image from object file



Cosa dovrebbe essere in grado di fare il compilatore per ottimizzare il codice?

Le **ottimizzazioni indipendenti dalla macchina (MI)** si basano sulla struttura logica del programma e non dipendono dall'hardware su cui verrà eseguito.

Tra queste c'è l'eliminazione del codice senza effetto (che non influenza il comportamento osservabile del programma / i risultati finali), la specializzazione dei calcoli (sostituire operazioni costose con versioni + economiche), l'eliminazione di calcoli ridondanti..

Tra le **ottimizzazioni machine-dependent (MD)** vi è il trarre vantaggio da specifiche caratteristiche hardware, gestire/nascondere la latenza, e gestire le risorse limitate della macchina (usare in modo efficiente i registri della CPU).

Models of Programs

Il codice sorgente è una rappresentazione goffa e troppo implicita per un compilatore che deve ottimizzare il programma. Per questo motivo, i compilatori traducono il codice in

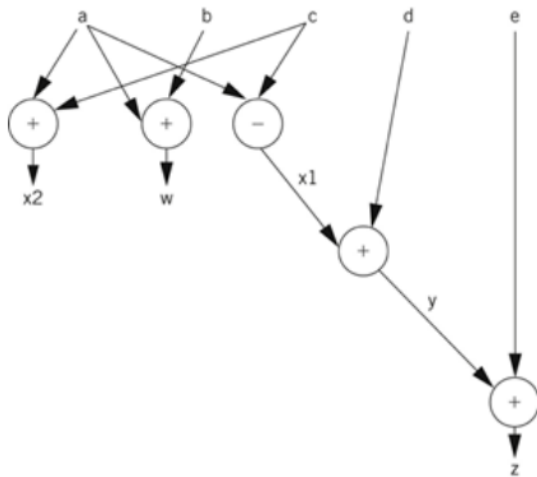
Rappresentazioni Intermedie per poterlo analizzare e ottimizzare. Il modello fondamentale utilizzato è il **Control/Data Flow Graph(CDFG)**.

Un data flow graph è un modello che rappresenta una porzione di programma privo di istruzioni condizionali (un **basic block**), ovvero una sequenza di codice con un solo punto di ingresso e un solo punto di uscita.

Single-Assignment Form: Per costruire un DFG efficace, il codice viene spesso riscritto in una forma ad "assegnazione singola". In codice normale una variabile x può essere scritta più volte : $x = a+b \dots x = c-d\dots$). Nel DFG, ogni volta che una variabile viene riassegnata, viene creato un nuovo nome univoco ($x_1, x_2\dots$), in modo che ogni variabile nel grafo sia prodotta da una sola operazione.

$w = a + b;$		$w = a + b;$
$x = a - c;$		$x_1 = a - c;$
$y = x + d;$		$y = x_1 + d;$
$x = a + c;$		$x_2 = a + c;$
$z = y + e;$		$z = y + e;$

Grazie alla forma a assegnazione singola, il DFG risulta essere un grafo **aciclico** (senza loop interni ai dati), questo è fondamentale per permettere agli algoritmi di analisi di funzionare correttamente.



Il DFG rende esplicito l'ordine necessario delle operazioni (ordinamento parziale):

- Se l'operazione B usa il risultato di A , allora A deve venire prima di B
- Se due operazioni non hanno dipendenze di dati tra loro, il compilatore capisce che possono essere riordinate o eseguite in parallelo senza cambiare risultato finale.

Partial order:

$a+c, a+b, a-c; x_1+d; y+e$

-
- Poiché un programma contiene decisioni e cicli, il DFG da solo non basta, allora entra in gioco il **CDFG** che combina i DFG con le strutture di controllo.

Il CDGF ha due tipi di nodi: i Decision nodes che rappresentano le scelte (if(cond)) e i data flow nodes che sono i basic block denotati dai DFG. Il compilatore usa il CDGF per navigare nella struttura logica del programma.

Machine Independent Optimization (MI)

1) Eliminare il codice che non ha effetto

Per trovare le porzioni di codice inutili e quindi eliminabili il compilatore si baserà sul CDGF. L'Algoritmo di pulizia funziona così:

1. Il compilatore prima identifica le **operazioni critiche**. Queste sono le istruzioni che hanno un effetto visibile all'esterno o essenziale: ad esempio tutte le operazioni di Input/Output (printf), scritture su memoria globale/non locale, istruzioni che impostano il valore di ritorno di una funzione.
2. Si tracciano a ritroso tutte le istruzioni necessarie per calcolare i dati usati da queste operazioni critiche
3. Tutto ciò che non è marcato come necessario in questo processo viene considerato "morto" ed eliminato.

Nell'esempio sotto si vede come gli assegnamenti ad "a" sono inutili perché non viene né stampato né usato per ritornare dalla funzione.

Initial Code:

```
int Dead(void) {  
    int a=10;  
    int z=50;  
    int c;  
    c=z*5;  
    printf(c);  
    a=20;  
    a=a*10; //No need of These Two Lines  
    return 0;  
}
```

Optimized Code:

```
int Dead(void) {  
    int a=10;  
    int z=50;  
    int c;  
    c=z*5;  
    printf(c);  
    return 0;  
}
```

Un'altra cosa che si può fare è eliminare anche il codice **irraggiungibile**, infatti analizzando il CFG si possono trovare blocchi che non potranno mai essere eseguiti perché non esiste un percorso logico che porti a essi (con gli if ed else). Ad esempio se ho if(0)... il compilatore eliminerà il blocco if dal binario finale.

Altra cosa da fare è **identificare il flusso di controllo inutile**:

- Se un if/else porta allo stesso risultato o la condizione è nota a tempo di compilazione, la ramificazione viene rimossa (**fusione dei redundant branches**)
- Se un blocco Bi è vuoto, viene rimosso dal grafo
- Se un blocco Bi è seguito sempre e solo da un blocco Bj, i due blocchi vengono fusi in uno solo
- **Branch Hoisting**: Se un salto punta a un blocco vuoto che a sua volta salta a un altro blocco, il compilatore modifica il primo salto per puntare direttamente alla destinazione finale, saltando l'intermediario.

Un'altra cosa utile per rimuovere codice senza effetto è la rimozione di operazioni che sono identità algebriche (es: $x = x + 0$, $x = x * 1$).

2) Specializzazione delle computazioni

L'obiettivo è sostituire operazioni costose o generiche con versioni più semplici e veloci.

Una prima tecnica è la riduzione della forza dell'operatore (**Operator Strength Reduction**).

Questa operazione sostituisce un'operazione matematica costosa con una o più operazioni economiche che producono lo stesso risultato.

Sostituzione della moltiplicazione: Viene trasformata in addizione o shift: ad esempio $x \times 2$ diventa $x + x \ll 1$ (shift a sinistra).

Sostituzione della divisione: Viene trasformata in sottrazione o shift (ad esempio $x/2$ diventa $x \gg 1$).

La strength reduction è una tecnica applicabile anche all'interno dei cicli per semplificare i calcoli sugli indici degli array.

Una seconda tecnica è la **propagazione delle costanti**:

Il compilatore analizza il codice per vedere se il valore è noto a tempo di compilazione. Se una variabile ha un valore noto, il compilatore sostituisce la variabile direttamente con il numero.

Constant folding: Se $\text{int } a=2$, e dopo c'è $b=a \times 10$, il compilatore calcola direttamente 20 e inserisce l'istruzione $b=20$ nel codice macchina, evitando la delega della moltiplicazione al processore durante l'esecuzione.

Ottimizzazione peephole: Riconoscere sequenze di istruzioni assembly che possono essere sostituite con una singola istruzione o con più veloci offerte da quel set di istruzioni specifico.

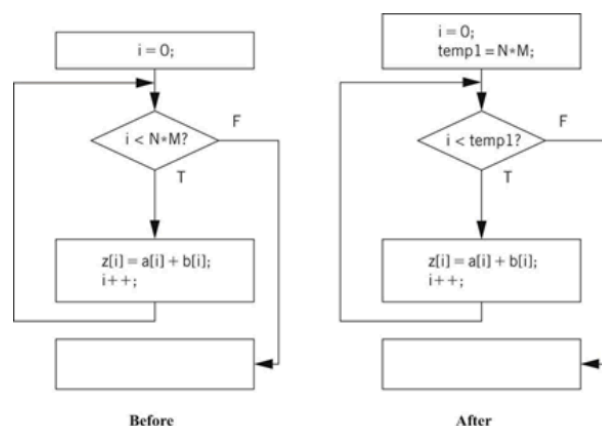
3) Code motion

Se il risultato di un'operazione all'interno di un ciclo non dipende da valori che cambiano durante l'iterazione del ciclo stesso ("loop-invariant"), allora non è necessario ricalcolarlo ogni volta.

Quindi il compilatore trova codice loop-invariant e lo sposta fuori dal loop, subito prima dell'ingresso nel ciclo.

In questo esempio $N \times M$ viene spostato fuori così faccio moltiplicazione solo una volta.

```
for (i=0; i<N*M; i++)  
    z[i] = a[i] + b[i];
```



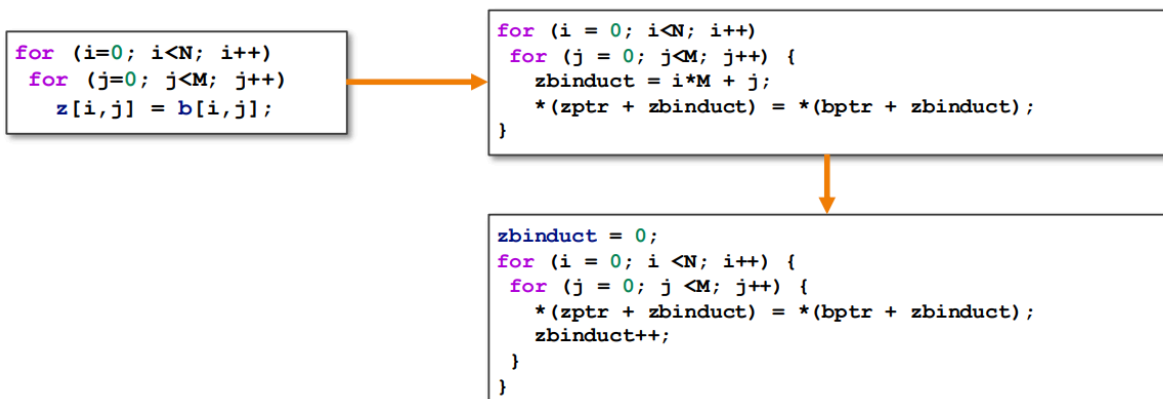
4) Induction Variable Elimination

Una variabile di induzione è una variabile il cui valore è derivato direttamente dal valore della variabile di iterazione del ciclo (il contatore del loop). Spesso il programmatore non scrive esplicitamente queste variabili, è il compilatore che le introduce per calcolare gli indirizzi di memoria necessari per accedere agli array.

Per esempio se scorro una matrice 2D, per accedere all'elemento $z[i][j]$, il processore deve calcolare l'indirizzo di memoria esatto:

IndirizzoBase + $(i \times M + j) \times \text{DimensioneElemento}$

Se questo calcolo viene eseguito a ogni iterazione, il processore spreca un botto di cicli a moltiplicare. Quindi si può creare prima la variabile e poi solo incrementarla di 1 a ogni ciclo e ho l'offset giusto

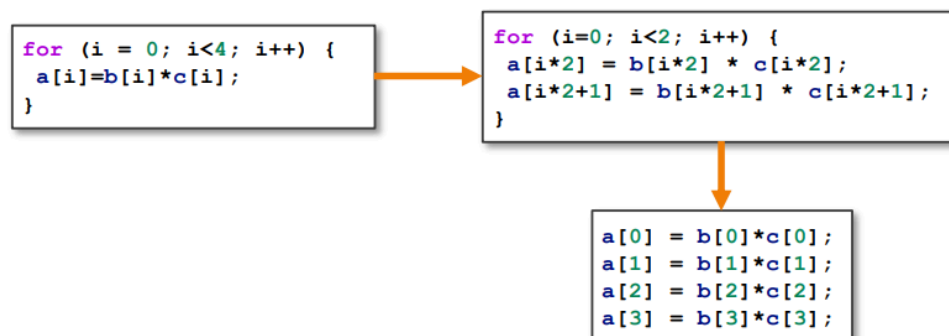


4) Enabling Transformations

Trasformazioni eseguite dal compilatore per ristrutturare il codice e facilitare ulteriori ottimizzazioni.

Loop unrolling:

Tecnica che consiste nel replicare il corpo del ciclo più volte per ridurre il numero totale di iterazioni necessarie per completare il lavoro. Il compilatore srotola il ciclo (duplica il body ad esempio) eseguendo più operazioni per ogni controllo. In questo modo riduce l'overhead del ciclo (istruzioni di controllo e di salto), ma aumenta la Code Size.



Loop fusion:

Tecnica che combina due o più cicli adiacenti in un unico ciclo.

La condizione necessaria per applicarla è che i cicli devono iterare sullo stesso intervallo di valori da 0 a N e i cicli non devono avere dipendenze tra i loro dati che impediscano la fusione. Il vantaggio è ridurre overhead (solo un contatore, solo un controllo).

```
for (i=0; i<N; i++)  
    a[i] = b[i] * 5;  
  
for (j=0; j<N; j++)  
    w[j] = c[j] * d[j];
```

```
for (i=0; i<N; i++) {  
    a[i] = b[i] * 5;  
    w[i] = c[i] * d[i];  
}
```

Loop Unswitching (Estrazione della condizione)

Questa tecnica sposta le decisioni condizionali al di fuori del ciclo se la condizione non cambia durante il ciclo. In questo modo si evita di valutare la stessa condizione if ad ogni iterazione.

```
for (i=0; i<N; i++) {  
    if(x>y)  
        a[i] = b[i] * x;  
    else  
        a[i] = b[i] * y;  
}
```

```
if(x>y){  
    for (i=0; i<N; i++)  
        a[i] = b[i] * x;  
} else {  
    for (i=0; i<N; i++)  
        a[i] = b[i] * y;  
}
```

Renaming:

Tecnica che assegna un nome univoco a ogni valore calcolato/definito nel codice, così l'ottimizzatore può riconoscere se un valore è già stato calcolato e ancora valido(live)..

a = x + y

b = x + y

a = 17

c = x + y

a₀ = x₀ + y₀

b₀ = x₀ + y₀

a₁ = 17

c₀ = x₀ + y₀

No need to
compute.

Machine-Dependent Optimizations (MD)

Sono quelle ottimizzazioni che si applicano sfruttando la conoscenza specifica dell'architettura hardware del processore target.

1) Sfruttare caratteristiche hardware:

Si può fare instruction pre-fetch e branch prediction, sfruttando i circuiti del processore che caricano le istruzioni in anticipo o prevedono l'esito dei salti condizionali per evitare stalli nella pipeline.

Si può anche preferire l'**esecuzione condizionale**, utilizzando istruzioni che vengono eseguite solo se una condizione è vera, eliminando i salti costosi.

Infine si possono usare modalità di indirizzamento avanzate complesse offerte dall'ISA per calcolare gli indirizzi di memoria in meno istruzioni.

2)Gestire la latenza:

Se un'operazione richiede dati dalla memoria (che è lenta), il compilatore ripianifica altre operazioni indipendenti per essere eseguite mentre la CPU attende l'arrivo dei dati, nascondendo così il ritardo.

Cache locality: Riorganizzare l'ordine delle iterazioni dei cicli per accedere alla memoria in modo sequenziale e prevedibile, sfruttando meglio la cache. Si può modificare il layout dei dati per migliorare la probabilità che i dati necessari si trovino già nella cache veloce.

3)Gestire le risorse limitate della macchina:

I registri della CPU sono la memoria più veloce, il compilatore deve decidere intelligentemente quali variabili mantenere sui registri e quali versare in RAM

I principali ostacoli all'ottimizzazione del compilatore:

1)Scope delle variabili eccessivo:

L'uso eccessivo di variabili globali o statiche al posto di variabili locali è uno dei freni maggiori per l'ottimizzatore. Il problema è che il compilatore deve assumere che qualsiasi funzione chiamata possa accedere o modificare una variabile globale.

Le variabili globali e statiche sono storate permanentemente nella memoria, e non riusabili nello spazio dello stack. Il compilatore non può mantenere queste variabili nei registri della CPU, ed è costretto a scrivere il valore in memoria prima di chiamare una funzione, e la funzione deve ricaricarlo dalla memoria per usarlo.

2)La parola chiave volatile:

L'uso del qualificatore volatile dice esplicitamente al compilatore di non ottimizzare l'accesso a quella variabile. Si usa per le variabili che possono cambiare esternamente al flusso del programma (come registri hardware modificati da periferiche o variabili globali condivise con una ISR). L'effetto è che il compilatore è costretto a rileggere la variabile dalla memoria ogni volta che viene usata, anche se il codice sembra non modificarla.

Esempio di problema quando non uso volatile:

Assumiamo che `buffer_full` sia una variabile che può essere modificata da un ISR, ossia possa cambiare asincronamente rispetto al flusso del programma:

```
int buffer_full;
int read_stream(void) {
    int count = 0;
    while (!buffer_full)
    {
        count++;
    }
    return count;
}
```

Siccome `buffer_full` non è dichiarato volatile, il compilatore assume che non venga modificato dall'esterno, quindi evita di ricaricarla dalla memoria e il risultato è un **infinite loop** perchè `buffer_full` rimane zero.

Promozione automatica delle espressioni aritmetiche:

Quando un'espressione coinvolge tipi di dati diversi il compilatore applica delle regole di conversione. Le regole di ANSI C standard dicono :

- Se un operando è `long double`, l'altro viene promosso a `long double`.
- Se un operando è `float`, l'altro viene promosso a `float`.
- Altrimenti, i tipi interi più piccoli (`char`, `short`) vengono generalmente promossi a `int`.

Esempio di cosa succede:

```
float f;
char c;
int r;

r = f * c;
```

- Call routine to convert c to float
- Call routine to perform floating point multiply with f
- Call routine to convert result from floating point to integer
- store result in r

Queste conversioni si pagano con un overhead sia in termini di tempo di esecuzione sia di dimensione del codice.

Una cosa simile succede quando nel mio codice ho chiamate a funzione che non hanno un prototipo prima della chiamata: in questo caso il compilatore applica le promozioni standard degli argomenti: `char` e `short` diventano `int` → **Prototipare sempre le funzioni.**

Compiler Optimization Levels e.g. Arm Compiler

- **Optimizing for code vs. speed**
 - **-Otime:** the compiler aggressively optimizes for time e.g. aggressive inlining, at the expense of a possible increase in image (binary) size.
 - **-Ospace:** the compiler optimizes for image size at the expense of a possible increase in execution time.
- **Optimization levels:**
 - **-O0 :** Minimum optimization. Turns off most optimizations. Best possible debug view.
 - **-O1 :** Restricted optimization e.g. removes unused inline functions. Turns off optimizations that seriously degrade the debug view.
 - **-O2 :** High optimization. e.g. compiler automatically inlines functions if optimizing for time. The compiler may perform optimizations that cannot be described by debug information.
 - **-O3 :** Maximum optimization e.g. loop unrolling if optimizing for time. This can give significant performance benefits at a small code size cost, but at the risk of a longer build time.

Come possiamo capire se il compilatore ha applicato le ottimizzazioni?

Code profiling: si possono utilizzare i **Code Profilers**, per misurare l'impatto delle ottimizzazioni. Questi strumenti misurano la frequenza e il tempo di esecuzione di specifiche istruzioni, l'uso della memoria e gli eventi.

Ci sono **Profiler Statistici** come Oprofile, VTune o Streamline, e poi ci sono strumenti di analisi dinamica come **Valgrind**.

Come possiamo modificare il source code per aiutare il compilatore a ottimizzare?

Il processo iterativo di base è : Leggere il manualaccio del compilatore -> applicare i suggerimenti e le direttive per ottimizzare -> guardare i risultati ottenuti -> se non soddisfatti ripetere il ciclo.

Ottimizzare per la potenza e per l'energia

Optimizing for power = mira a ridurre la quantità di energia consumata in un piccolo intervallo di tempo.

Optimizing for energy = mira a ottimizzare la quantità totale di energia consumata.

La potenza è legata alla dissipazione del calore, l'energia invece è il totale che si paga alla fine di tutto, e decide la durata della batteria.

Relazione critica tra potenza e energia: E' possibile avere profili di potenza diversi che risultano nella stessa energia totale, oppure consumare meno energia totale utilizzando un budget di potenza più elevato per un tempo molto più breve (ad esempio eseguire un calcolo velocemente e poi andare in sleep mode).

Code size vs Energia/Potenza

Esiste una relazione tra la dimensione del codice e il consumo energetico:

ridurre la Code Size riduce l'uso della memoria, il che generalmente riduce il consumo.

Tuttavia, tecniche come il Loop Unrolling o l'Inlining aumentano la dimensione del codice, ma eliminano l'overhead di gestione del loop o delle chiamate a funzione, riducendo il tempo di esecuzione e quindi l'energia totale. Bisogna trovare il giusto compromesso.

Tante volte ridurre la code size risulta nell'avere più operazioni di calcolo costose che a loro volta consumano potenza ed energia!

L'ottimizzazione a volte ferisce la manutenibilità del codice.

High Level Optimizations

Queste richiedono che il programmatore capisca la logica del problema per ristrutturarlo in modo efficiente.

Principio chiave : **Do Less Work.**

Lazy execution: Non calcolare un dato fino a che non è strettamente necessario

Early decisions: Prova a usare se possibile risultati intermedi per prendere una decisione logica.

Ottimizzazione algoritmica:

Bisogna saper scegliere l'algoritmo giusto: sostituisce la ricerca lineare in una ricerca binaria ad esempio

Ottimizzazione dell'accesso alla memoria:

Questo è cruciale per il consumo di energia. Le operazioni di trasferimento dalla memoria sono le più costose: 10 o 100 volte più costose in termini di energia rispetto a un'operazione aritmetica. Quindi bisogna organizzare i dati e le istruzioni per massimizzare l'uso dei registri e della cache, riducendo al minimo i viaggi verso la RAM o la Flash.

Gestione della precisione:

Bisogna bilanciare la precisione matematica con la velocità e il consumo:

se il microcontrollore non ha una Floating Point Unit, ogni operazione float viene emulata via software ed è lentissima. Usare double a 64 bit consuma più memoria e banda passante rispetto a float. Se possibile, convertire i calcoli in virgola fissa.

20) Embedded Real Time Systems

Embedded OS: Sistema operativo progettato per essere usato in sistemi embedded, che gestisce risorse limitate come memoria, I/O e clocks..

Deve essere molto **compatto e energy-efficient**.

Embedded RTOS: La sua caratteristica chiave è la **prevedibilità**. Deve rispondere agli eventi in modo rapido e all'interno di un ritardo garantito. Elabora i dati in cicli deterministici.

Sistemi reattivi

La maggior parte dei sistemi embedded sono reattivi, ovvero reagiscono a eventi esterni. In un contesto real-time, essere reattivi significa fornire una risposta entro un tempo predefinito (deadline).

Mentre gli OS general-purpose allocano risorse cercando di essere equi e senza garanzie temporali (possono missare le deadlines), gli RTOS allocano le risorse specificamente per **soddisfare le scadenze temporali**.

Processes and Operating Systems

Molte applicazioni moderne sono sofisticate. Scriverle come singolo programma sequenziale le rende troppo complesse da gestire (anche perchè poi con i vincoli temporali diventa ancora più complesso) → un unico loop infinito diventa ingestibile.

Per risolvere questo problema si utilizza l'**astrazione del Processo (o Task)**, ossia l'applicazione viene decomposta in piccoli pezzi funzionali indipendenti chiamati Task.

Il ruolo del RTOS è gestire questi task, ossia commutare rapidamente tra un task e l'altro per soddisfare i requisiti temporali di ciascuno.

Ogni task ha il suo stato : registri / memoria ecc..., e i tasks runnano sullo stesso spazio di indirizzi (come i threads)

Perché usare i task?

Esempio: Motore di Compressione Dati

Si ha un sistema che riceve dati da una linea seriale, li comprime e li invia.

Problema: Questo è un sistema multi-rate: i dati arrivano a una certa velocità, la compressione richiede un tempo variabile, e l'invio avviene a un'altra velocità ancora. L'input è **asincrono** (non sappiamo quando arriveranno i caratteri).

Necessità dei task: Se si tentasse di gestire tutto in un unico ciclo, le strutture di controllo diventerebbero enormemente complesse per non perdere dati in ingresso mentre si sta comprimendo o inviando. Viene naturale dividere il lavoro in task separati (uno che legge, uno che comprime, uno che invia).

Osservazione: Il Real-time garantisce che un processo venga completato in un intervallo di tempo definito, ma non garantisce nulla sulla durata totale o sulla velocità di processamento.

Release time: è definito come l'istante di tempo in cui un task diventa pronto (ready) per essere eseguito, ossia quando entra nella coda dei processi pronti e si mette in attesa di essere schedulato.

Deadline: l'istante di tempo entro il quale una computazione deve essere terminata.

- **Deadline Periodica :** Si applica ai task che si ripetono ciclicamente. In questo caso la deadline è fissata a un tempo specifico relativo al periodo del task. La deadline può essere un momento diverso dalla fine del periodo stesso.
- **Deadline aperiodica:** Si applica ai task che rispondono a eventi asincroni (che non hanno una periodicità fissa). In questo caso la deadline viene misurata come un intervallo di tempo a partire dal Release Time.

Timing Violations:

Jitter: è definito come la **variabilità** del tempo necessario per completare un task. In un sistema ideale, un task impiegherebbe sempre lo stesso tempo; nei sistemi reali questo tempo fluttua, creando incertezza.

Cosa succede se il task non finisce entro la deadline?

- **Hard Deadline:** se questa viene mancata, il sistema è considerato **fallito**(ad esempio nei sistemi safety-critical mancare una deadline può mettere in pericolo vite umane).
- **Soft Deadline:** Se la deadline viene mancata, l'utente potrebbe notarlo, ma il sistema non fallisce e può continuare

RTOS Scheduling

- Various scheduling algorithms exist:
 - **Cooperative scheduling:** no pre-emption, tasks can end themselves in a cooperative manner
 - **Preemptive scheduling:** tasks can be interrupted by other tasks of higher priorities
 - **Earliest deadline first approach:** the task with the earliest deadline is served first

Negli event-driven RTOS, lo scheduling è preemptive, perchè se si verifica un evento più importante di quello in esecuzione il sistema operativo deve poterlo interrompere.

Problemi dello scheduling:

Riusciamo a soddisfare tutte le deadlines?Quanta CPU power ci serve?

Lo stesso processo di scheduling consuma tempo per la CPU.

Introduction to Embedded RTOS:

L'OS deve tenere traccia delle priorità dei task, dello stato di scheduling e degli activation record dei task. I task possono essere creati sia staticamente(prima che il sistema si avvii) oppure dinamicamente(durante l'esecuzione).

Embedded vs general purpose scheduling:

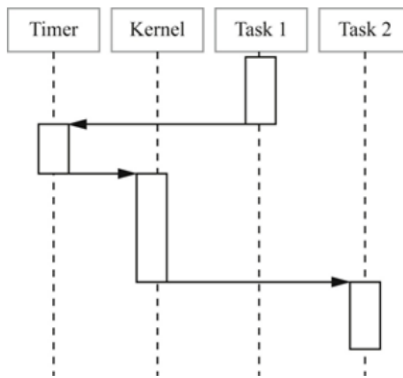
Mentre negli OS normali si punta soprattutto alla fairness(dare un pò di CPU a tutti), nei sistemi embedded la priorità è eseguire entro le deadlines, quindi **i processi a bassa priorità non dovrebbero runnare per tanto tempo.**

Il modo più affidabile per soddisfare questi vincoli temporali è progettare un **preemptive operating system.**

Preemptive scheduling:

In qualsiasi momento la CPU deve essere assegnata al task pronto con la priorità maggiore, anche a costo di interrompere forzatamente un task in esecuzione.

Ruolo del timer: Un interrupt periodico del timer (Tick) restituisce il controllo al Kernel del sistema operativo(è lui che decide quale task far runnare). La durata del periodo del timer è chiamata "Time quantum" ed è il più piccolo incremento con cui è possibile controllare l'attività della CPU. L'obiettivo è evitare che i processi a bassa priorità monopolizzino la CPU per troppo tempo.



Context switch:

Meccanismo che permette alla CPU di passare da eseguire un task all'altro. Poiché non ci si può affidare a semplice codice C per farlo, questo processo richiede l'uso del linguaggio assembly.

Contesto di un task = l'insieme dei registri della CPU che descrivono lo stato corrente del task.

Quando il kernel decide di cambiare task, salva lo stato del task corrente nello stack del task stesso o in una struttura dati dedicata. Il Kernel poi determina quale task eseguire dopo. I valori dei registri del nuovo stack vengono prelevati dal suo stack e caricati nella CPU. Questo include l'aggiornamento dello Stack Pointer per puntare alla memoria del nuovo task.

Il context switch è costoso e introduce un overhead.

RTOS Scheduling Algorithms

Rate Monotonic Scheduling (RMS):

L'RMS è una politica di scheduling a **priorità** fissa, utilizzata nei RTOS per gestire **processi periodici**.

Rate Monotonic = deriva dal fatto che la priorità assegnata a un task è una funzione monotona della sua frequenza (rate) di esecuzione.

Più breve è il periodo, più alta è la priorità. I processi che devono essere eseguiti più frequentemente hanno priorità massima.

Assunzioni: assumiamo che le priorità sono assegnate staticamente e non cambiano durante l'esecuzione. Supponiamo che i task siano periodici (si ripetono a intervalli regolari). Assumiamo che **deadline = periodo** ossia il task deve terminare entro la fine del suo periodo.

Assumiamo che il tempo di esecuzione del task sia costante e si conosca.

Assumiamo che non ci sono dipendenze di dati tra i processi.

Si assume che il tempo per il context switch sia zero.

Come funziona: E' preemptive, se c'è un processo pronto con priorità più alta di quello che sta girando allora questo viene interrotto subito. Bisogna costruire una timeline lunga tanto quanto il m.c.m tra i periodi dei processi.

Problema: non c'è un assegnamento flessibile delle priorità che garantisce di non eccedere la capacità disponibile della CPU (magari il periodo totale è 12 ma devo metterne 15).

Utilizzo CPU: Non permette di usare al 100% la CPU

Implementazione RMS: Si sorta l'array dei processi in ordine di periodo, si scansiona sto array, se vedo che un processo è pronto e ha priorità + alta lo si sceglie.

Earliest Deadline First Scheduling (EDF)

Ha un schema **dinamico** di assegnazione delle priorità.

Il processo più vicino alla sua deadline ha la priorità + alta.

Richiede il ricalcolo delle priorità a ogni timer interrupt.

Riesce a usare il **100% della CPU**, MA potrebbe fallire nel stare nella deadline.

Generalmente è troppo costoso da usare in pratica.

Implementazione di EDF:

L'implementazione è molto + complessa: non possiamo presortare i processi in un array.

Per evitare di risortare i records a ogni cambio, possiamo creare un **albero binario**, che mantiene le deadline sempre sortate e incrementalmente aggiorna il sort.

Noi aggiorneremo le deadlines a ogni tick del timer così:

Attraversiamo tutto l'albero e diminuiamo il tempo alla deadline, eliminiamo la priorità più alta e la riassegniamo all'albero.

Cosa fare se il set di processi non è schedulabile?

Si cambiano i requisiti sulle deadlines, oppure si riduce il tempo di esecuzione dei processi, oppure si piglia na CPU più veloce.

System Performance:

Abbiamo assunto di conoscere il tempo di esecuzione dei processi. Ma in realtà il tempo di esecuzione non è costante, però si può trovare il worst-case e usare quello.

In alcuni casi il ritardo dato dal context-switching può essere parecchio: il tempo che richiede un context switch dipende da quanto contesto deve essere salvato e dal tempo di esecuzione dello scheduler (ad esempio EDF è $O(n \log n)$).

Inoltre v'è considerata anche la **interrupt latency**:

E' il tempo che passa da quando un interrupt viene richiesto a quando il servizio è completato. Questo tempo dipende dalla latenza di interrupt del processore, dal tempo esecuzione dell'interrupt handler, dai ritardi dovuti allo scheduling RTOS.

Un interrupt di alta priorità potrebbe ritardarne uno di bassa priorità. E' buona pratica dividere l'interrupt handling in 2 pezzi di codice:

l'ISR esegue solo le operazioni minime per rispondere al dispositivo, il thread in user-mode invece fa il resto.

FreeRTOS Examples:

Qui un processo è chiamato Task. I task vengono creati dinamicamente con `xTaskCreate()`.

Un task è implementato come una funzione che contiene un ciclo `while(1)`, perchè non dovrebbe mai ritornare come una normale funzione.

I task possono essere bloccati con `vTaskDelay()` o cancellati con `vTaskDelete()`.

Una volta creati i task, il sistema operativo inizia a girare chiamando `vTaskStartScheduler()`.

Algoritmo di Scheduling:

FreeRTOS usa un algoritmo di scheduling specifico chiamato **Fixed Priority Preemptive Scheduling with Time Slicing**.

Ossia lo scheduler non modifica dinamicamente le priorità assegnate ai task.

Time slicing: Se ci sono più task pronti con la stessa priorità, il tempo di esecuzione della CPU viene diviso equamente tra loro (Round Robin)

Risorse Condivise e Race Conditions:

In un sistema multitasking come FreeRTOS, i task spesso devono condividere risorse (memoria globale e periferiche I/O).

Race condition: Si verifica quando il risultato di un'operazione dipende dall'ordine imprevedibile con cui i task accedono alla risorsa → provoca corruzione / inconsistenza nei dati.

Sezione critica: è la sezione di codice che non deve essere interrotto da altri task che accedono alla stessa risorsa.

Per prevenire le race condizioni e implementare le sezioni critiche, abbiamo bisogno di **oggetti kernel**, come i semafori, le message queues, i segnali e le pipes.

Semafori:

Sono oggetti kernel che uno o più task possono **acquisire** o **rilasciare**.

Protocollo:

Il task chiama wait() e attende di ottenere il semaforo, se è libero lo ottiene altrimenti si blocca.

Il task esegue le operazioni della sezione critica.

Il task rilascia il semaforo con post().

Per implementare la wait() e la post() il bus del microprocessore deve supportare un'operazione di lettura/scrittura atomica.

Mutex:

E' un speciale tipo di semaforo binario: i suoi stati sono unlocked(0) o locked(1).

Il mutex a differenza di un semaforo ha un proprietario: solo l'owner può fare unlock.

Invece un semaforo binario normale non ha owner e chiunque può rilasciare il semaforo.

Priority inversion:

L'uso di risorse condivise può causare un problema critico di scheduling chiamato Priority Inversion:

Scenario:

- Un task a bassa priorità blocca una risorsa condivisa prendendo il mutex
- Un task ad alta priorità diventa pronto, preemta quello con con bassa priorità, ma trova la risorsa occupata -> si blocca
- Lo scheduler passa il controllo a quello a priorità bassa, che rilascia il mutex
- Però arriva un task a media priorità (che non usa la risorsa) che preemta quello a bassa priorità ed esegue lui prima di quello ad alta.

FreeRTOS risolve la priority inversion con la **priority inheritance**:

Quando un task ad alta priorità è bloccato in attesa di un Mutex detenuto da un task a bassa priorità, il sistema operativo alza temporaneamente la priorità del task a bassa priorità al livello di quella del task ad alta. Questo impedisce ai task di media priorità di interrompere quelli a bassa priorità, permettendogli così di finire la sezione critica e rilasciare il Mutex più velocemente possibile.

Message queues: Agiscono come un buffer FIFO disaccoppiando il task che invia dati da quello che li riceve.

Gli interrupt vengono mascherati durante l'esecuzione di una sezione critica -> lunghe sezioni critiche possono causare problemi.